



MP3 Decoder

Programmer's Guide

For HiFi DSPs



Cadence Design Systems, Inc.
2655 Seely Ave.
San Jose, CA 95134
www.cadence.com

© 2024 Cadence Design Systems, Inc. All rights reserved.
Cadence Design Systems, Inc. (Cadence), 2655 Seely Ave., San Jose, CA 95134, USA.

Trademarks: Trademarks and service marks of Cadence Design Systems, Inc. (Cadence) contained in this document are attributed to Cadence with the appropriate symbol. For queries regarding Cadence's trademarks, contact the corporate legal department at the address shown above or call 1-800-862-4522.

All other trademarks are the property of their respective holders.

Restricted Print Permission: This publication is protected by copyright and any unauthorized use of this publication may violate copyright, trademark, and other laws. Except as specified in this permission statement, this publication may not be copied, reproduced, modified, published, uploaded, posted, transmitted, or distributed in any way, without prior written permission from Cadence. This statement grants you permission to print one (1) hard copy of this publication subject to the following conditions:

- The publication may be used solely for personal, informational, and noncommercial purposes;
- The publication may not be modified in any way;
- Any copy of the publication or portion thereof must include all original copyright, trademark, and other proprietary notices and this permission statement,
- The information contained in this document cannot be used in the development of like products or software, whether for internal or external use, and shall not be used for the benefit of any other party, whether or not for consideration, and
- Cadence reserves the right to revoke this authorization at any time, and any such use shall be discontinued immediately upon written notice from Cadence.

Disclaimer: Information in this publication is subject to change without notice and does not represent a commitment on the part of Cadence. The information contained herein is the proprietary and confidential information of Cadence or its licensors, and is supplied subject to, and may be used only by Cadence's customer in accordance with, a written agreement between Cadence and its customer. Except as may be explicitly set forth in such agreement, Cadence does not make, and expressly disclaims, any representations or warranties as to the completeness, accuracy or usefulness of the information contained in this document. Cadence does not warrant that use of such information will not infringe any third party rights, nor does Cadence assume any liability for damages or costs of any kind that may result from use of such information.

Restricted Rights: Use, duplication, or disclosure by the Government is subject to restrictions as set forth in FAR52.227-14 and DFAR252.227-7013 et seq. or its successor.
For further assistance, contact Cadence Online Support at <https://support.cadence.com/>.

Version: 3.15

Last Updated: August 2024

Cadence Design Systems, Inc.
2655 Seely Ave.
San Jose, CA 95134 www.cadence.com

Contents

Contents	iii
Figures	iv
Tables	v
1. Introduction to the HiFi MP3 Decoder	1
1.1 MP3 Description	1
1.2 Document Overview	2
1.3 HiFi MP3 Decoder Specifications	2
1.4 HiFi MP3 Decoder Performance	3
1.4.1 MP2 Decoder Memory	4
1.4.2 MP2 Decoder Timings	4
1.4.3 MP3 Decoder Memory	4
1.4.4 MP3 Decoder Timings	5
1.4.5 MP3 5.1-channel Decoder Memory	5
1.4.6 MP3 5.1-channel Decoder Timings	5
2. Generic HiFi Audio Codec API	7
2.1 Memory Management	7
2.1.1 API Object	8
2.1.2 API Memory Table	8
2.1.3 Memory Types	8
2.1.3.1 Persistent Memory	8
2.1.3.2 Scratch Memory	8
2.1.3.3 Input Buffer	8
2.1.3.4 Output Buffer	8
2.2 C Language API	9
2.3 About Generic API Errors	10
2.4 Commands	11
2.4.1 Start-up API Stage	12
2.4.2 Set Codec-specific Parameters Stage	12
2.4.3 Memory Allocation Stage	12
2.4.4 Initialize Codec Stage	13
2.4.5 Get Codec-specific Parameters Stage	14
2.4.6 Codec Process Stage	14
2.5 Files Describing the API	15
2.6 HiFi API Command Reference	16
2.6.1 Common API Errors	16
2.6.2 XA_API_CMD_GET_LIB_ID_STRINGS	17
2.6.3 XA_API_CMD_GET_API_SIZE	20
2.6.4 XA_API_CMD_INIT	21

2.6.5	XA_API_CMD_GET_MEMTABS_SIZE	25
2.6.6	XA_API_CMD_SET_MEMTABS_PTR	26
2.6.7	XA_API_CMD_GET_N_MEMTABS.....	27
2.6.8	XA_API_CMD_GET_MEM_INFO_SIZE	28
2.6.9	XA_API_CMD_GET_MEM_INFO_ALIGNMENT	29
2.6.10	XA_API_CMD_GET_MEM_INFO_TYPE.....	30
2.6.11	XA_API_CMD_GET_MEM_INFO_PRIORITY	31
2.6.12	XA_API_CMD_SET_MEM_PTR.....	33
2.6.13	XA_API_CMD_INPUT_OVER	34
2.6.14	XA_API_CMD_SET_INPUT_BYTES.....	35
2.6.15	XA_API_CMD_GET_CURIDX_INPUT_BUF	36
2.6.16	XA_API_CMD_EXECUTE	37
2.6.17	XA_API_CMD_GET_OUTPUT_BYTES	40
2.6.18	XA_API_CMD_GET_CONFIG_PARAM	41
2.6.19	XA_API_CMD_SET_CONFIG_PARAM	43
3.	HiFi DSP MP3 Decoder	44
3.1	Files Specific to the MP3 Decoder	45
3.2	Configuration Parameters	45
3.3	Usage Notes.....	47
3.4	MP3 Decoder-specific Commands.....	48
3.4.1	Initialization and Process Errors.....	48
3.4.2	XA_API_CMD_SET_CONFIG_PARAM	51
3.4.3	XA_API_CMD_GET_CONFIG_PARAM	57
4.	Introduction to the Example Testbench	74
4.1	Making Executable Files	74
4.2	Usage	75
4.3	ID3 Decoding Example	77
5.	References	79

Figures

Figure 1	HiFi Audio Codec Interfaces	7
Figure 2	API Command Sequence Overview	11
Figure 3	Flowchart for MP3 Decoder Integration	44

Tables

Table 2-1 Codec API.....	9
Table 2-2 Error Codes Format	10
Table 2-3 Commands for Initialization	12
Table 2-4 Commands for Setting Parameters	12
Table 2-5 Commands for Initial Table Allocation	13
Table 2-6 Commands for Memory Allocation.....	13
Table 2-7 Commands for Initialization	14
Table 2-8 Commands for Getting Parameters	14
Table 2-9 Codec Processing Commands	15
Table 2-10 XA_CMD_TYPE_LIB_NAME Subcommand	17
Table 2-11 XA_CMD_TYPE_LIB_VERSION Subcommand	18
Table 2-12 XA_CMD_TYPE_API_VERSION Subcommand	19
Table 2-13 XA_API_CMD_GET_API_SIZE Command	20
Table 2-14 XA_CMD_TYPE_INIT_API_PRE_CONFIG_PARAMS Subcommand.....	21
Table 2-15 XA_CMD_TYPE_INIT_API_POST_CONFIG_PARAMS Subcommand	22
Table 2-16 XA_CMD_TYPE_INIT_PROCESS Subcommand.....	23
Table 2-17 XA_CMD_TYPE_INIT_DONE_QUERY Subcommand	24
Table 2-18 XA_API_CMD_GET_MEMTABS_SIZE Command	25
Table 2-19 XA_API_CMD_SET_MEMTABS_PTR Command	26
Table 2-20 XA_API_CMD_GET_N_MEMTABS Command	27
Table 2-21 XA_API_CMD_GET_MEM_INFO_SIZE Command	28
Table 2-22 XA_API_CMD_GET_MEM_INFO_ALIGNMENT Command.....	29
Table 2-23 XA_API_CMD_GET_MEM_INFO_TYPE Command	30
Table 2-24 Memory Type Indices	30
Table 2-25 XA_API_CMD_GET_MEM_INFO_PRIORITY Command.....	31
Table 2-26 Memory Priorities	32
Table 2-27 XA_API_CMD_SET_MEM_PTR Command.....	33
Table 2-28 XA_API_CMD_INPUT_OVER Command	34
Table 2-29 XA_API_CMD_SET_INPUT_BYTES Command.....	35
Table 2-30 XA_API_CMD_GET_CURIDX_INPUT_BUF Command	36
Table 2-31 XA_CMD_TYPE_DO_EXECUTE Subcommand.....	37
Table 2-32 XA_CMD_TYPE_DONE_QUERY Subcommand	38
Table 2-33 XA_CMD_TYPE_DO_RUNTIME_INIT Subcommand	39
Table 2-34 XA_API_CMD_GET_OUTPUT_BYTES Command	40
Table 2-35 XA_CONFIG_PARAM_CUR_INPUT_STREAM_POS Subcommand	41
Table 2-36 XA_CONFIG_PARAM_GEN_INPUT_STREAM_POS Subcommand	42
Table 2-37 XA_CONFIG_PARAM_CUR_INPUT_STREAM_POS Subcommand	43

Table 3-1	XA_MP3DEC_CONFIG_PARAM_PCM_WDSZ Subcommand	51
Table 3-2	XA_MP3DEC_CONFIG_PARAM_CRC_CHECK Subcommand	52
Table 3-3	XA_MP3DEC_CONFIG_PARAM_MCH_ENABLE Subcommand	53
Table 3-5	XA_MP3DEC_CONFIG_PARAM_ACTIVATE_VLC_REWIND Subcommand	55
Table 3-6	XA_MP3DEC_CONFIG_PARAM_NONSTD_STRM_SUPPORT Subcommand	56
Table 3-7	XA_MP3DEC_CONFIG_PARAM_SAMP_FREQ Subcommand	57
Table 3-8	XA_MP3DEC_CONFIG_PARAM_NUM_CHANNELS Subcommand	58
Table 3-9	XA_MP3DEC_CONFIG_PARAM_CHMODE_INFO Subcommand	59
Table 3-10	XA_MP3DEC_CONFIG_PARAM_BITRATE Subcommand	60
Table 3-11	XA_MP3DEC_CONFIG_PARAM_PCM_WDSZ Subcommand	61
Table 3-12	XA_MP3DEC_CONFIG_PARAM_MCH_STATUS Subcommand	62
Table 3-13	XA_MP3DEC_CONFIG_PARAM_EXTN_PRESENT Subcommand	63
Table 3-14	XA_MP3DEC_CONFIG_PARAM_LFE_PRESENT Subcommand	64
Table 3-15	XA_MP3DEC_CONFIG_PARAM_NUM_XCHANS Subcommand	65
Table 3-16	XA_MP3DEC_CONFIG_PARAM_CHAN_CONFIG Subcommand	66
Table 3-17	XA_MP3DEC_CONFIG_PARAM_CHAN_MAP Subcommand	67
Table 3-18	XA_MP3DEC_CONFIG_PARAM_DAB_MP2 Subcommand	68
Table 3-19	XA_MP3DEC_CONFIG_PARAM_ORIGINAL_OR_COPY Subcommand	69
Table 3-20	XA_MP3DEC_CONFIG_PARAM_COPYRIGHT_FLAG Subcommand	70
Table 3-21	XA_MP3DEC_CONFIG_PARAM_MCH_EXT_HDR_INFO Subcommand	71
Table 3-22	XA_MP3DEC_CONFIG_PARAM_MCH_COPYRIGHT_ID_PTR Subcommand	72

Document Change History

Version	Changes
3.7	- History section added. - Added memory and timing performance data for HiFi Mini and HiFi 3.
3.8	Updated memory and timing data.
3.9	Added memory and timing data for HiFi 4.
3.10	Added performance data for HiFi 3z in Section 1.4.
3.11	Added performance data for Fusion F1 in Section 1.4.
3.12	Added performance data for HiFi 5 in Section 1.4.
3.13	- Added performance and memory data for HiFi 1 in Section <u>1.4</u>. - Updates in Section <u>3.4</u>.
3.14	Added performance and memory data in Section <u>1.4</u>.
3.15	Added performance and memory data in Section 1.4.

1. Introduction to the HiFi MP3 Decoder

The HiFi MP3 Decoder implements the features for the audio codec, which are specified by MPEG-1, MPEG-2, and MPEG-2.5 standards. MPEG-1 and MPEG-2 standards originate from the Moving Picture Experts Group (MPEG) (a working group of ISO/IEC ^{[1][2]}).

Note MPEG-2.5 is a proprietary extension to MPEG-1/2 Layer III by the Fraunhofer Institute for Integrated Circuits (FhG IIS), Germany.

The HiFi MP3 Decoder is a general term for three different levels of MPEG-1/2/2.5 audio technology supplied by Cadence® Tensilica®, as separate libraries. The libraries are as follows:

- **MP2 Decoder library (mp2_dec):** Implements MPEG-1/2 Layers I and II 2-channel decoding.
- **MP3 Decoder library (mp3_dec):** Implements MPEG-1/2/2.5 Layers I, II, and III 2-channel decoding.
- **MP3 5.1-channel Decoder library (mp3mch_dec):** Implements MPEG-1/2/2.5 Layers I, II, and III 2-channel decoding as well as MPEG-2 Layer II 5.1-channel decoding.

1.1 MP3 Description

Quote from the Moving Picture Experts Group:

"MPEG-1 Audio Layer III, also known as MP3, has been implemented in manifold ways. Many software packages exist to rip a track from a CD Audio and compress it in MP3. This has given rise to innovative ways of consuming music, such as the ability to create compilations to one's liking that can then be downloaded to light non-mechanical MP3 players. With the arrival of MP3, the music world has been changed without recognition."

MP3 was further extended to MPEG-2 to support sampling rates and encoded bit rates. The first of the MP3 players arrived in 1998. This success was due to the compression achieved with respect to the CD format.

The target of 1.33 bits per sample results in a bitstream of 128kb/s for stereo 48 kHz audio, which is 11.7 times more compressed than uncompressed PCM Audio.

The codec handles audio signals sampled between 8 kHz and 48 kHz. It operates on a frame of 1152 samples (Layers II and III) or 384 samples (Layer I).

1.2 Document Overview

This document covers all the information required to integrate the HiFi Audio Codecs into an application. The HiFi Codec libraries implement a simple API to encapsulate the complexities of the coding operations and simplify the application and system implementation.

- Section 1 describes the HiFi MP3 decoder in brief.
- Section 2 describes parts of the API that are common to all the HiFi codecs.
- Section 3 describes all the features and information particular to the HiFi MP3 Decoder.
- Section 4 describes an example testbench.
- Section 5 lists the references.

1.3 HiFi MP3 Decoder Specifications

The Cadence® Tensilica® HiFi DSP MP3 Decoder is a full-precision MP3 decoder that supports MPEG-1 and MPEG-2 Layer I, II, and III and MPEG-2.5 Layer III audio streams. It implements the following features in conformance to the MPEG-1, MPEG-2, and MPEG 2.5 specifications.

Cadence Audio Codec API is used in all three libraries.

MPEG-1

- Compliant with ISO/IEC 11172-3 standard
 - Decoding of MPEG-1 Layers I and II
 - Decoding of MPEG-1 Layer III (`mp3_dec` and `mp3mch_dec`, only)
- Sample rates: 32, 44.1, and 48 kHz
- Bit rates: 32-320 Kbps for Layer III
- Bit rates: 32-384 Kbps for Layer II
- Bit rates: 32-448 Kbps for Layer I
- Supports free format bit-rate decoding for Layer III
- Mono and stereo channels
- Joint stereo (MS and Intensity Stereo)
- Supports VBR
- Supports CRC check for Layers I and II
- Full precision decoder

MPEG-2

Features in addition to MPEG-1:

- Compliant with ISO/IEC 13818-3 standard.
 - Main audio channels (Lo/Ro) decoding of MPEG-2 Layers I and II
 - Main audio channels (Lo/Ro) decoding of MPEG-2 Layer III (`mp3_dec` and `mp3mch_dec`, only)
 - 5.1-channel decoding of MPEG-2 Layer II (`mp3mch_dec` only)
- Additional sample rates: 16, 22.05, and 24 kHz
- Additional bit rates: 8 -160 Kbps for Layer III applicable to lower sample rates
- Additional bit rates: 8 -160 Kbps for Layer II applicable to lower sample rates
- Additional bit rates: 32-256 Kbps for Layer I applicable to lower sample rates
- Bit rates up to 1067 Kbps for Layer II 5.1-channel decoding
- Supports VBR

MPEG-2.5

Features in addition to MPEG-2:

- Decoding of MPEG-2.5 Layer III (`mp3_dec` and `mp3mch_dec`, only)
- Sample rates: 8, 11.025, and 12 kHz
- Bit rates: 8-160 kbps for Layer III
- Supports VBR

Note	In addition to the above-mentioned standard features, the Cadence Tensilica MP3 decoder may support other common sampling rates and bit-rate combinations under each layer. Those are not part of the conformance test.
-------------	---

1.4 HiFi MP3 Decoder Performance

The Cadence Tensilica's HiFi DSP MP3 Decoder is characterized on the 5-stage HiFi DSP core. For design purposes, use memory usage and performance metrics.

- The API structure size returned by `XA_API_CMD_GET_API_SIZE` is approximately 50 bytes.
- The memory table structure size returned by `XA_API_CMD_GET_MEMTABS_SIZE` is approximately 80 bytes.

1.4.1 MP2 Decoder Memory

For memory specifications of the MP2 Decoder, see Table 1-1 and Table 1-2:

Table 1-1 MP2 Decoder Library Memory

Text (Kbytes)					Data (Kbytes)
HiFi 1	HiFi 3	HiFi 3z	HiFi 4	HiFi 5	
12.2	11.9	11.9	12.2	13.5	4.0

Table 1-2 MP2 Decoder Run-time Memory

Run-time Memory (Kbytes)				
Persistent	Scratch	Stack	Input	Output
7.3	1.9	0.4	2.0	4.5

1.4.2 MP2 Decoder Timings

For timing specifications of the MP2 Decoder, see Table 1-3:

Table 1-3 MP2 Decoder Timings

Rate kHz	Channels	Bit Rate kbps	Average CPU Load (MHz)				
			HiFi 1	HiFi 3	HiFi 3z	HiFi 4	HiFi 5
48	2	192	3.5	3.5	3.1	2.8	2.6

1.4.3 MP3 Decoder Memory

For memory specifications of the MP3 Decoder, see Table 1-4 and Table 1-5:

Table 1-4 MP3 Decoder Library Memory

Text (Kbytes)					Data (Kbytes)
HiFi 1	HiFi 3	HiFi 3z	HiFi 4	HiFi 5	
24.1	23.9	24.4	25.9	29.8	14.9

Table 1-5 MP3 Decoder Run-time Memory

Run-time Memory (Kbytes)				
Persistent	Scratch	Stack	Input	Output
12.4	7.0	0.8	2.0	4.5

1.4.4 MP3 Decoder Timings

For timing specifications of the MP3 Decoder, see Table 1-6:

Table 1-6 MP3 Decoder Timings

Rate kHz	Channels	Bit Rate kbps	Average CPU Load (MHz)				
			HiFi 1	HiFi 3	HiFi 3z	HiFi 4	HiFi 5
44.1	2	128	3.7	3.4	3.1	2.8	2.5
44.1	2	320	4.5	4.2	3.9	3.6	3.1
48	2	128	3.8	3.6	3.3	2.9	2.6
48	2	320	4.9	4.5	4.2	3.8	3.4

1.4.5 MP3 5.1-channel Decoder Memory

For memory specifications of the MP3 5.1-channel Decoder, see Table 1-7 and Table 1-8:

Table 1-7 MP3 5.1-channel Decoder Library Memory

Text (Kbytes)					Data (Kbytes)
HiFi 1	HiFi 3	HiFi 3z	HiFi 4	HiFi 5	
41.3	39.8	41.8	43.3	48.2	15.7

Table 1-8 MP3 5.1-channel Decoder Run-time Memory

Run-time Memory (Kbytes)				
Persistent	Scratch	Stack	Input	Output
21.6	36.5	1.2	4.0	27

1.4.6 MP3 5.1-channel Decoder Timings

For timing specifications of the MP3 5.1-Channel Decoder, see Table 1-9:

Table 1-9 MP3 5.1-channel Decoder Timings

Rate kHz	Channels	Bit Rate kbps	Average CPU Load (MHz)				
			HiFi 1	HiFi 3	HiFi 3z	HiFi 4	HiFi 5
48	6	1.66	18.2	20.4	16.3	15.5	14.2

Note	Performance specification measurements are carried on a cycle-accurate simulator assuming an ideal memory system, that is, one with zero memory wait states. This is equivalent to running with all code and data in local memories or using an infinite-size, pre-warmed cache model.
Note	The MCPS numbers for HiFi 3z/HiFi 4 are obtained by running the test with the binary that is recompiled from the HiFi 3 optimized source code in the HiFi 3z/HiFi 4 configuration. No specific optimization is done for HiFi 3z/HiFi 4.
Note	The MCPS numbers for HiFi 5 are obtained by running the test with the binary that is recompiled from the HiFi 1 optimized source code in HiFi 5 configuration. No specific optimization is done for HiFi 5.
Note	The output of the HiFi 3/HiFi 3z/HiFi 4 code is not bit-exact with that of the HiFi 1/HiFi 5 optimized code.
Note	Output buffer requirements are specified for 16-bit output, which is typically used. For 24-bit output, the output buffer size is doubled.
Note	All the memory and MCPS numbers are captured with the XT-CLANG compiler and the RJ-2024.3 tools.

2. Generic HiFi Audio Codec API

This section provides information about the API that is common to all the HiFi audio codec libraries. The API facilitates any codec that works in the overall method shown in Figure 1.

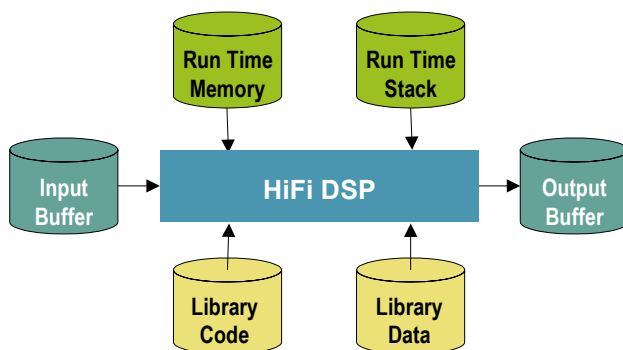


Figure 1 HiFi Audio Codec Interfaces

Section 2.1 discusses all the types of run-time memory required by the codecs. As no state information is held in static memory, a single thread can perform time division processing of multiple codecs. Additionally, multiple threads can perform concurrent codec processing. The API is implemented in a way that the application does not need to consider the codec implementation.

The codec requests the minimum sizes required for the input and output buffers through the API. Hence, before running the codec command, the codec requires that the input buffer is filled with data up to the minimum size for the input buffer. However, the codec may not consume all of the data in the input buffer. In order to properly manage the input data, the application needs to monitor the amount of data used, transfer any remaining unused data to the lower part of the input buffer, and then proceed to fill the remaining space in the buffer with new data until it reaches the minimum size again. The codec produces data in the output buffer. After the codec operation, the output data must be removed from the output buffer.

Applications that use these libraries should not make any assumptions about the size of the PCM “chunks” of data that each call to a codec produces or consumes. Normally, the chunks are the exact size of the underlying frame of the specified codec algorithm, and they will vary between codecs and also between different operating modes of the same codec. The application should provide enough data to fill the input buffer. However, some codecs provide information after the initialization stage to adjust the number of bytes of PCM data they need.

2.1 Memory Management

The HiFi audio codec API supports a flexible memory scheme and a simple interface that simplifies the integration into the final application. The API allows the codecs to request the memory required for their operations during run time.

The run-time memory requirement consists primarily of scratch and persistent memory. The codecs also require an input buffer and output buffer to transfer data into and out of the codec.

2.1.1 API Object

For each API call, the codec API stores its data in a small structure that is passed via a pointer handle to an opaque object from the application. The codec references all state information and the memory tables from this structure.

2.1.2 API Memory Table

During the memory allocation, the application is prompted to allocate memory for each of the following memory areas (described in Section 2.1.3). The reference pointer to each memory area is stored in this memory table, and the reference to this memory table is stored in the API object.

2.1.3 Memory Types

2.1.3.1 Persistent Memory

This type of memory is referred to as static or context memory. It consists of the state or historical information that is preserved from one codec invocation to the next within the same thread or instance. The codecs assume that the persistent memory remains unchanged by the system, except for the codec library itself, for the entire duration of the codec operation.

2.1.3.2 Scratch Memory

This type of temporary buffer is used by the codec for processing. The contents of this memory region should be unchanged if the codec processing is active, that is, if the thread running the codec is inside any API call. The system can use this region freely between successive calls to the codec.

2.1.3.3 Input Buffer

This is the buffer that the algorithm uses for accepting input data. Before the call to the codec, the input buffer must be completely filled with input data.

2.1.3.4 Output Buffer

This output buffer is used by algorithm to write the output. Before the algorithm completes its tasks, the output buffer must be accessible for the codec. The application can modify the output buffer pointer between codec calls, enabling the codec to write directly to the designated output area. It's important to note that the codec will never write more data than the requested size of the output buffer.

2.2 C Language API

A single interface function is used to access the codec, with the operation specified by command codes. The API C call is defined for each codec library and is specified in the codec-specific section. Each library has a single C API call.

The C parameter definitions for every codec library are the same and are specified in the following table:

Table 2-1 Codec API

xa_<codec>	
Description	This C API is the only function that allows accessing the audio codec.
Syntax	<pre>XA_ERRORCODE xa_<codec>(xa_codec_handle_t p_xa_module_obj, WORD32 i_cmd, WORD32 i_idx, pVOID pv_value);</pre>
Parameters	p_xa_module_obj Pointer to the opaque API structure. i_cmd Command. i_idx Command subtype or index. pv_value Pointer to the variable used to pass in or retrieve properties from the state structure.
Returns	Returns an error code to indicate success or failure of the API call.

The types used for the C API calls are defined in the supplied header files as:

```
typedef signed int      WORD32;
typedef void            *pVOID;
```

Each time the C API for the codec is called, a pointer to a privately allocated data structure is passed as the first argument. This argument is treated as an opaque handle, as the application does not require the data to be looked at within the structure. The size of the structure is supplied by a specific API command so that the application can allocate the required memory. Do not use `sizeof()` for the type of the opaque handle.

Some command codes are further divided into subcommands. The command and its subcommand are passed to the codec via the second and third arguments respectively.

When a value is passed to or returned from an API command, the expected or returned value is passed through a pointer given as the fourth argument to the C API function. When passing a pointer value to the codec, the pointer is cast to `pVOID`. In these cases, it is incorrect to pass a pointer to a pointer. For example, when the application passes the codec a pointer to an allocated memory region.

Because the operations required to decode or encode audio streams are similar, the HiFi DSP API enables the application to use the same set of procedures for each stage. By maintaining a pointer to the single API function and passing the correct API object, the same code base can be used to carry out the operations required for any of the supported codecs.

2.3 About Generic API Errors

The API functions return the error code of `XA_ERRORCODE` type, which is `signed int`. Table 2-2 defines the format of the error codes.

Table 2-2 Error Codes Format

31	30-15	14 – 11	10 – 6	5 – 0
Fatal	Reserved	Class	Codec	Sub code

The format of error codes is explained as follows:

- The errors that are returned from the APIs are divided into fatal and nonfatal errors based on severity. Fatal errors require restarting the codec, while nonfatal errors provide information to the application.
- Errors can be further classified into the following three classes:
 - API errors occur when the API is used incorrectly.
 - Config errors occur when the codec parameters are incorrect or not supported.
 - Process Errors occur during the primary encoding or decoding and indicate issues that have arisen due to invalid or bad input data.
- The Sub code specifies the exact condition that triggers the error.

2.4 Commands

This section details the commands associated with the flow chart shown in Figure 2. The white blocks in the flow chart represent a common set of generic API commands, and the yellow blocks represent codec-specific commands.

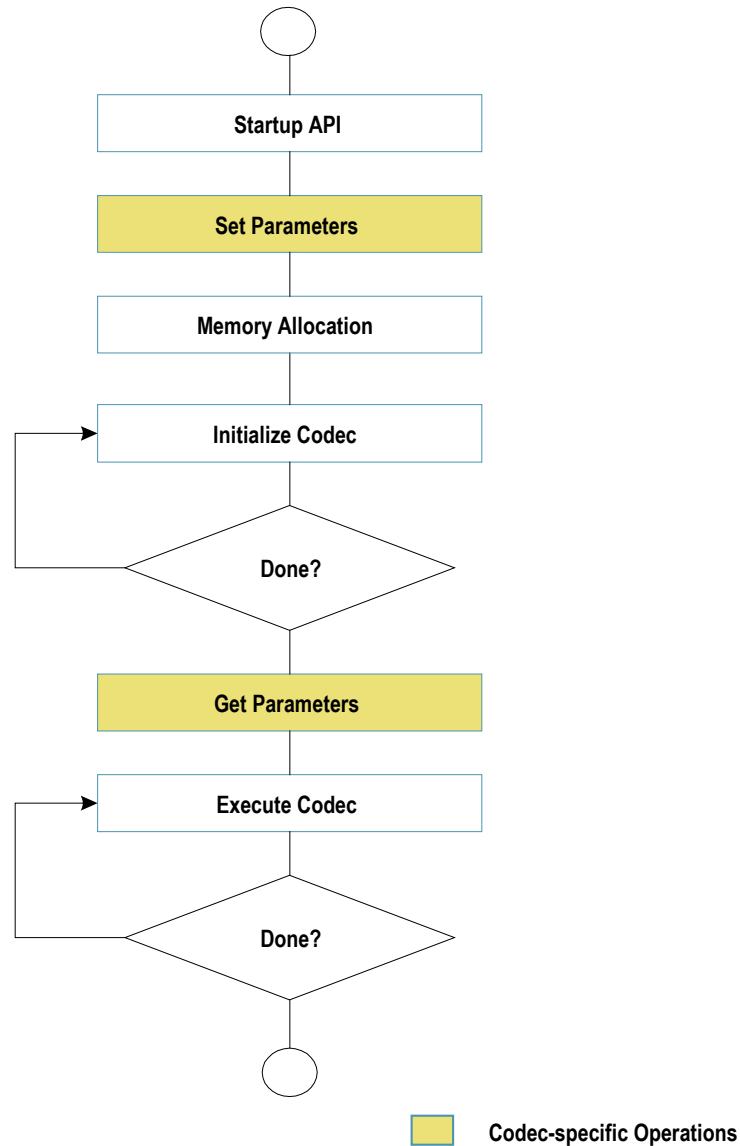


Figure 2 API Command Sequence Overview

The following sections list the commands in the order they should occur for each stage of the flow shown in Figure 2. For individual commands, definitions, and examples, refer to Section 2.6.

2.4.1 Start-up API Stage

The commands listed in Table 2-3 should be run once each during start-up. The commands to get the various identification strings from the codec library are optional and for information only. The command to get the API object size is mandatory as the real object type is hidden in the library, and therefore there is no type available to use with `sizeof()`.

Table 2-3 Commands for Initialization

Command / Subcommand	Description
XA_API_CMD_GET_LIB_ID_STRINGS XA_CMD_TYPE_LIB_NAME	Gets the name of the library.
XA_API_CMD_GET_LIB_ID_STRINGS XA_CMD_TYPE_LIB_VERSION	Gets the version of the library.
XA_API_CMD_GET_LIB_ID_STRINGS XA_CMD_TYPE_API_VERSION	Gets the version of the API.
XA_API_CMD_GET_API_SIZE	Gets the size of the API structure.
XA_API_CMD_INIT XA_CMD_TYPE_INIT_API_PRE_CONFIG_PARAMS	Sets the default values of all the configuration parameters.

2.4.2 Set Codec-specific Parameters Stage

Refer to the specific codec section for the parameters that can be set. These parameters either control the encoding process or determine the output format of the decoder PCM data.

Table 2-4 Commands for Setting Parameters

Command / Subcommand	Description
XA_API_CMD_SET_CONFIG_PARAM XA_<codec>_CONFIG_PARAM_<param_name>	Sets codec-specific parameters. See the codec-specific section for parameter definitions.

2.4.3 Memory Allocation Stage

The commands listed in Table 2-5 should be run once, only after all the codec-specific parameters are set. After the application allocates the memory table structure (MEMTABS) per the specified size, a pointer to the memory table structure is passed to the codec API. After the codec-specific parameters are set, the initial codec setup is completed by performing the post-configuration portion of the initialization to determine the initial operating mode of the codec and assign sizes to the blocks of memory required for its operation. The application then requests a count of the number of memory blocks.

Table 2-5 Commands for Initial Table Allocation

Command / Subcommand	Description
XA_API_CMD_GET_MEMTABS_SIZE	Gets the size of the memory structures to be allocated for the codec tables.
XA_API_CMD_SET_MEMTABS_PTR	Passes the memory structure pointer allocated for the tables.
XA_API_CMD_INIT XA_CMD_TYPE_INIT_API_POST_CONFIG_PARAMS	Calculates the required sizes for all the memory blocks based on the codec-specific parameters.
XA_API_CMD_GET_N_MEMTABS	Obtains the number of memory blocks required by the codec.

The commands listed in Table 2-6 should then be run in a loop to allocate the memory. The application first requests all the attributes of the memory block and then allocates it, making sure to abide by the alignment requirements. Finally, the pointer to the allocated block of memory is passed back through the API. For the input and output buffers, it is not necessary to assign the correct memory at this point. However, the input and output buffer locations must be assigned before their first use in the Process stage. The type field refers to the memory blocks, such as input or persistent, as described in Section 2.1.

Table 2-6 Commands for Memory Allocation

Command / Subcommand	Description
XA_API_CMD_GET_MEM_INFO_SIZE	Gets the size of the memory type being referred to by the index.
XA_API_CMD_GET_MEM_INFO_ALIGNMENT	Gets the alignment information of the memory type being referred to by the index.
XA_API_CMD_GET_MEM_INFO_TYPE	Gets the type of memory referenced by the index.
XA_API_CMD_GET_MEM_INFO_PRIORITY	Gets the allocation priority of memory being referred to by the index.
XA_API_CMD_SET_MEM_PTR	Sets the pointer to the memory allocated for the referred index to the input value.

2.4.4 Initialize Codec Stage

The commands listed in Table 2-7 should be run in a loop during initialization. These commands should be called until the initialization is completed, as indicated by the `XA_CMD_TYPE_INIT_DONE_QUERY` command. In general, decoders can loop multiple times until the header information is found. However, encoders will perform exactly one call before they signal they are done.

There is a major difference between encoding Pulse Code Modulated (PCM) data and decoding stream data. During the initialization of a decoder, the initialization task reads the input stream to discover the parameters of the encoding. However, for an encoder, PCM data has no header information. Even so, the encoder application is still required to perform the initialization described in this stage. However, encoders

will not consume data during initialization. Furthermore, this implies that some encoders provide parameters that can be used to modify the input buffer data requirements after the initialization stage. These modifications will always be a reduction in the size. The application only needs to provide the reduced amount for each of the main codec processes.

In general, the application will signal the codec the number of bytes available in the input buffer and signal whether it is the last iteration. It is not normal to reach the end of the data during initialization. Still, if a decoder is presented with a corrupt stream, it will allow a smooth termination. After the codec initialization is called, the application will ask for the number of bytes consumed. The application can also ask if the initialization is complete. It is advisable to always ask, even in encoders requiring only a single pass. A decoder application must keep iterating until the initialization is completed.

Table 2-7 Commands for Initialization

Command / Subcommand	Description
XA_API_CMD_SET_INPUT_BYTES	Sets the number of bytes available in the input buffer for initialization.
XA_API_CMD_INPUT_OVER	Signals the codec at the end of the bitstream.
XA_API_CMD_INIT XA_CMD_TYPE_INIT_PROCESS	Searches for the valid header, decodes header to get the parameters, and initializes state and configuration structures.
XA_API_CMD_INIT XA_CMD_TYPE_INIT_DONE_QUERY	Checks if the initialization process has been completed.
XA_API_CMD_GET_CURIDX_INPUT_BUF	Gets the number of input buffer bytes consumed by the last initialization.

2.4.5 Get Codec-specific Parameters Stage

After initialization, the codec can supply the application with information. In the case of decoders, these would be the parameters it has extracted from the encoded header in the stream.

Table 2-8 Commands for Getting Parameters

Command / Subcommand	Description
XA_API_CMD_GET_CONFIG_PARAM XA_<codec>_CONFIG_PARAM_<param_name>	Gets the parameter value from the codec. See the codec-specific section for parameter definitions.

2.4.6 Codec Process Stage

The commands listed in Table 2-9 should be run continuously until the data is exhausted or the application wants to terminate the process. This is similar to the initialization stage but includes support for the management of the output buffer. After each iteration, the application requests for the amount of data written to the output buffer. This amount is always limited by the size of the buffer that was requested during the memory block allocation. To alter the output buffer position, use XA_API_CMD_SET_MEM_PTR with the output buffer index.

Table 2-9 Codec Processing Commands

Command / Subcommand	Description
XA_API_CMD_INPUT_OVER	Signals the end of the bitstream to the library.
XA_API_CMD_SET_INPUT_BYTES	Sets the number of bytes available in the input buffer for the codec processing.
XA_API_CMD_EXECUTE XA_CMD_TYPE_DO_EXECUTE	Runs the codec thread.
XA_API_CMD_EXECUTE XA_CMD_TYPE_DONE_QUERY	Checks if the end of the stream has been reached.
XA_API_CMD_GET_OUTPUT_BYTES	Gets the number of bytes output by the codec in the last frame.
XA_API_CMD_GET_CURIDX_INPUT_BUF	Gets the number of input buffer bytes consumed by the last call to the codec.

2.5 Files Describing the API

Following are the common include files (include):

- `xa_apicmd_standards.h`
Includes command definitions for the generic API calls.
- `xa_error_standards.h`
Includes macros and definitions for all the generic errors.
- `xa_memory_standards.h`
Includes definitions for the memory block allocation
- `xa_type_def.h`
Includes definitions of all datatypes required for the API calls

2.6 HiFi API Command Reference

This section describes different commands along with their associated subcommands. This section does not describe the codec-specific commands. The particular codec commands are the SET and GET commands for the operational parameters.

The commands are listed below in the following sections based on their primary command type (`i_cmd`). Each section contains a table for every subcommand. If subcommands are not available, one primary command is presented.

The commands are followed by an example C call code. Along with the call, a definition of the variable types used is provided. This is to avoid any confusion over the type of the fourth argument. The examples are not complete C code extracts, as the variables are not initialized before they are used.

The errors returned by the API are listed after each of the command definitions. However, few errors are common to all the API commands; these are listed in Section 2.6.1. The codec-specific sections will define all the errors possible from the codec-specific commands. For more information on Process errors, refer to Section 3.4.1.

2.6.1 Common API Errors

These errors are fatal and should not be encountered during normal application operations. They signal that a serious error has occurred in the application, calling the codec.

- `XA_API_FATAL_MEM_ALLOC`
`p_xa_module_obj` is NULL
- `XA_API_FATAL_MEM_ALIGN`
`p_xa_module_obj` is not aligned to 4 bytes
- `XA_API_FATAL_INVALID_CMD`
`i_cmd` is not a valid command
- `XA_API_FATAL_INVALID_CMD_TYPE`
`i_idx` is invalid for the specified command (`i_cmd`)

2.6.2 XA_API_CMD_GET_LIB_ID_STRINGS

Table 2-10 XA_CMD_TYPE_LIB_NAME Subcommand

Subcommand	XA_CMD_TYPE_LIB_NAME
Description	This command obtains the library's name in the form of a string. The maximum length of the string that the library will provide is 30 bytes. Therefore, the application shall pass a pointer to a buffer having a minimum size of 30 bytes. This command is optional.
Actual Parameters	p_xa_module_obj NULL i_cmd XA_API_CMD_GET_LIB_ID_STRINGS i_idx XA_CMD_TYPE_LIB_NAME pv_value process name - Pointer to a character buffer in which the name of the library is returned.
Restrictions	None

Note Codec object is not required as the name is static data in the codec library.

Example

```
char process_name[30];
res = (*api_func) (NULL,
    XA_API_CMD_GET_LIB_ID_STRINGS,
    XA_CMD_TYPE_LIB_NAME,
    (pVOID) process_name);
```

Errors

- XA_API_FATAL_MEM_ALLOC
This error is suppressed as p_xa_module_obj is NULL.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

Table 2-11 XA_CMD_TYPE_LIB_VERSION Subcommand

Subcommand	XA_CMD_TYPE_LIB_VERSION
Description	This command obtains the library version in the form of a string. The maximum length of the string that the library will provide is 30 bytes. Therefore, the application shall pass a pointer to a buffer having a minimum size of 30 bytes. This command is optional.
Actual Parameters	p_xa_module_obj NULL i_cmd XA_API_CMD_GET_LIB_ID_STRINGS i_idx XA_CMD_TYPE_LIB_VERSION pv_value lib_version - Pointer to a character buffer in which the version of the library is returned.
Restrictions	None

Note Codec object is not required as the version is static data in the codec library.

Example

```
char lib_version[30];
res = (*api_func) (NULL,
                  XA_API_CMD_GET_LIB_ID_STRINGS,
                  XA_CMD_TYPE_LIB_VERSION,
                  (pVOID) lib_version);
```

Errors

- XA_API_FATAL_MEM_ALLOC
This error is suppressed as p_xa_module_obj is NULL.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

Table 2-12 XA_CMD_TYPE_API_VERSION Subcommand

Subcommand	XA_CMD_TYPE_API_VERSION
Description	This command obtains the API version in the form of a string. The maximum length of the string that the library will provide is 30 bytes. Therefore, the application shall pass a pointer to a buffer having a minimum size of 30 bytes. This command is optional.
Actual Parameters	p_xa_module_obj NULL i_cmd XA_API_CMD_GET_LIB_ID_STRINGS i_idx XA_CMD_TYPE_API_VERSION pv_value api_version - Pointer to a character buffer in which the version of the API is returned.
Restrictions	None

Note Codec object is not required as the version is static data in the codec library.

Example

```
char api_version[30];
res = (*api_func) (NULL,
                  XA_API_CMD_GET_LIB_ID_STRINGS,
                  XA_CMD_TYPE_API_VERSION,
                  (pVOID) api_version);
```

Errors

- XA_API_FATAL_MEM_ALLOC
This error is suppressed as p_xa_module_obj is NULL.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

2.6.3 XA_API_CMD_GET_API_SIZE

Table 2-13 XA_API_CMD_GET_API_SIZE Command

Subcommand	None
Description	This command is used to obtain the size of the API structure and allocate memory for the API structure. The pointer to the API size variable is passed and the API returns the structure size in bytes. The API structure is used for the interface and is persistent.
Actual Parameters	p_xa_module_obj NULL i_cmd XA_API_CMD_GET_API_SIZE i_idx NULL pv_value &api_size – Pointer to the API size variable.
Restrictions	The application will allocate memory with an alignment of 4 bytes.

Note Codec object is not required as the size is fixed for the codec library.

Example

```
unsigned int api_size;
res = (*api_func) (NULL,
    XA_API_CMD_GET_API_SIZE,
    0,
    (pVOID) &api_size);
```

Errors

- XA_API_FATAL_MEM_ALLOC
This error is suppressed as p_xa_module_obj is NULL.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

2.6.4 XA_API_CMD_INIT

Table 2-14 XA_CMD_TYPE_INIT_API_PRE_CONFIG_PARAMS Subcommand

Subcommand	XA_CMD_TYPE_INIT_API_PRE_CONFIG_PARAMS
Description	This command sets the default value of the configuration parameters. The configuration parameters can then be altered by using one of the codec-specific parameter setting commands. Refer to Section 3.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_INIT i_idx XA_CMD_TYPE_INIT_API_PRE_CONFIG_PARAMS pv_value NULL
Restrictions	None

Example

```
res = (*api_func) (api_obj,  
                  XA_API_CMD_INIT,  
                  XA_CMD_TYPE_INIT_API_PRE_CONFIG_PARAMS,  
                  NULL);
```

Errors

- Common API Errors.

Table 2-15 XA_CMD_TYPE_INIT_API_POST_CONFIG_PARAMS Subcommand

Subcommand	XA_CMD_TYPE_INIT_API_POST_CONFIG_PARAMS
Description	This command calculates the sizes of all the memory blocks required by the application. It should occur after the codec-specific parameters are set.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_INIT i_idx XA_CMD_TYPE_INIT_API_POST_CONFIG_PARAMS pv_value NULL
Restrictions	None

Example

```
res = (*api_func)(api_obj,  
    XA_API_CMD_INIT,  
    XA_CMD_TYPE_INIT_API_POST_CONFIG_PARAMS,  
    NULL);
```

Errors

- Common API Errors.

Table 2-16 XA_CMD_TYPE_INIT_PROCESS Subcommand

Subcommand	XA_CMD_TYPE_INIT_PROCESS
Description	This command initializes the codec. In the case of a decoder, it searches for the valid header and performs the header decoding to get the encoded stream parameters. This command is part of the initialization loop. It must be repeatedly called until the codec signals it has finished. In the case of an encoder, the initialization of the codec is performed. No output data is created during initialization.
Actual Parameters	<code>p_xa_module_obj</code> <code>api_obj</code> – Pointer to API structure. <code>i_cmd</code> XA_API_CMD_INIT <code>i_idx</code> XA_CMD_TYPE_INIT_PROCESS <code>pv_value</code> NULL
Restrictions	None

Example

```
res = (*api_func) (api_obj,  
                  XA_API_CMD_INIT,  
                  XA_CMD_TYPE_INIT_PROCESS,  
                  NULL);
```

Errors

- Common API Errors.
- See the codec-specific section for Process Errors.

Table 2-17 XA_CMD_TYPE_INIT_DONE_QUERY Subcommand

Subcommand	XA_CMD_TYPE_INIT_DONE_QUERY
Description	This command checks to see if the initialization process has been completed. If it has, the flag value is set to 1; otherwise, it is set to zero. A pointer to the flag variable is passed as an argument.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_INIT i_idx XA_CMD_TYPE_INIT_DONE_QUERY pv_value &init_done – Pointer to a flag that indicates the completion of the initialization process.
Restrictions	None

Example

```
unsigned int init_done;
res = (*api_func) (api_obj,
                  XA_API_CMD_INIT,
                  XA_CMD_TYPE_INIT_DONE_QUERY,
                  (pVOID) &init_done);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

2.6.5 XA_API_CMD_GET_MEMTABS_SIZE

Table 2-18 XA_API_CMD_GET_MEMTABS_SIZE Command

Subcommand	None
Description	This command obtains the table size to hold the memory blocks required for the codec operation. The API returns the total size of the required table. A pointer to the size variable is sent with this API command, and the codec writes the variable's value.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_MEMTABS_SIZE i_idx NULL pv_value &proc_mem_tabs_size – Pointer to the memory size variable.
Restrictions	The application shall allocate memory with an alignment of 4 bytes.

Example

```

unsigned int proc_mem_tabs_size;
res = (*api_func) (api_obj,
                  XA_API_CMD_GET_MEMTABS_SIZE,
                  0,
                  (pVOID) &proc_mem_tabs_size);

```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

2.6.6 XA_API_CMD_SET_MEMTABS_PTR

Table 2-19 XA_API_CMD_SET_MEMTABS_PTR Command

Subcommand	None
Description	This command sets the memory structure pointer in the library to the allocated value.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_SET_MEMTABS_PTR i_idx NULL pv_value alloc – Allocated pointer.
Restrictions	The application will allocate memory with an alignment of 4 bytes.

Example

```
int * alloc; //alloc is a pointer to the allocated memory
res = (*api_func)(api_obj,
    XA_API_CMD_SET_MEMTABS_PTR,
    0,
    (pVOID) alloc);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.
- XA_API_FATAL_MEM_ALIGN
pv_value is not aligned to 4 bytes.

2.6.7 XA_API_CMD_GET_N_MEMTABS

Table 2-20 XA_API_CMD_GET_N_MEMTABS Command

Subcommand	None
Description	This command obtains the number of memory blocks needed by the codec. This value is used as the iteration counter for allocating the memory blocks. A pointer to each memory block will be placed in the previously allocated memory tables. The pointer to the variable is passed to the API and the codec writes the value for this variable.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_N_MEMTABS i_idx NULL pv_value &n_mems – Number of memory blocks required to be allocated.
Restrictions	None

Example

```
int n_mems;
res = (*api_func) (api_obj,
                  XA_API_CMD_GET_N_MEMTABS,
                  0,
                  (pVOID) &n_mems);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

2.6.8 XA_API_CMD_GET_MEM_INFO_SIZE

Table 2-21 XA_API_CMD_GET_MEM_INFO_SIZE Command

Subcommand	Memory index
Description	This command obtains the size of the memory type referred to by the index. The size in bytes is returned in the variable pointed to by the final argument. Note that this is the actual size needed and does not include any alignment packing space.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_MEM_INFO_SIZE i_idx Index of the memory pv_value &size – Pointer to the memory size.
Restrictions	None

Example

```
int index;
unsigned int size;
res = (*api_func) (api_obj,
                  XA_API_CMD_GET_MEM_INFO_SIZE,
                  index,
                  (pVOID) &size);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.
- XA_API_FATAL_INVALID_CMD_TYPE
i_idx is an invalid memory block number; valid block numbers obey the relation $0 \leq i_idx < n_mems$ (See XA_API_CMD_GET_N_MEMTABS).

2.6.9 XA_API_CMD_GET_MEM_INFO_ALIGNMENT

Table 2-22 XA_API_CMD_GET_MEM_INFO_ALIGNMENT Command

Subcommand	Memory index
Description	This command gets the alignment information of the memory type being referred to by the index. The alignment required in bytes is returned to the application.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_MEM_INFO_ALIGNMENT i_idx Index of the memory. pv_value &alignment – Pointer to the alignment info variable
Restrictions	None

Example

```
int index;
unsigned int alignment;
res = (*api_func)(api_obj,
    XA_API_CMD_GET_MEM_INFO_ALIGNMENT,
    index,
    (pVOID) &alignment);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.
- XA_API_FATAL_INVALID_CMD_TYPE
i_idx is an invalid memory block number; valid block numbers obey the relation $0 \leq i_idx < n_mems$ (See XA_API_CMD_GET_N_MEMTABS).

2.6.10 XA_API_CMD_GET_MEM_INFO_TYPE

Table 2-23 XA_API_CMD_GET_MEM_INFO_TYPE Command

Subcommand	Memory index
Description	This command gets the type of memory being referred to by the index.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_MEM_INFO_TYPE i_idx Index of the memory. pv_value &type – Pointer to the memory type variable.
Restrictions	None

Example

```
int index;
unsigned int type;
res = (*api_func)(api_obj,
    XA_API_CMD_GET_MEM_INFO_TYPE,
    index,
    (pVOID) &type);
```

Table 2-24 Memory Type Indices

Type	Description
XA_MEMTYPE_PERSIST	Persistent Memory
XA_MEMTYPE_SCRATCH	Scratch Memory
XA_MEMTYPE_INPUT	Input Buffer
XA_MEMTYPE_OUTPUT	Output Buffer

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.
- XA_API_FATAL_INVALID_CMD_TYPE
i_idx is an invalid memory block number; valid block numbers obey the relation $0 \leq i_idx < n_mems$ (See XA_API_CMD_GET_N_MEMTABS).

2.6.11 XA_API_CMD_GET_MEM_INFO_PRIORITY

Table 2-25 XA_API_CMD_GET_MEM_INFO_PRIORITY Command

Subcommand	Memory index
Description	This command gets the allocation priority of memory referred to by the index. (The meaning of the levels is defined on a codec-specific basis. Unless the codec defines it otherwise, this command returns a fixed dummy value.)
Actual Parameters	p_xa_module_obj api_obj - Pointer to API structure. i_cmd XA_API_CMD_GET_MEM_INFO_PRIORITY i_idx Index of the memory. pv_value &priority - Pointer to the memory priority variable.
Restrictions	None

Example

```
int index;
unsigned int priority;
res = (*api_func)(api_obj,
                  XA_API_CMD_GET_MEM_INFO_PRIORITY,
                  index,
                  (pVOID) &priority);
```

Table 2-26 Memory Priorities

Priority	Type
0	XA_MEMPRIORITY_ANYWHERE
1	XA_MEMPRIORITY_LOWEST
2	XA_MEMPRIORITY_LOW
3	XA_MEMPRIORITY_NORM
4	XA_MEMPRIORITY_ABOVE_NORM
5	XA_MEMPRIORITY_HIGH
6	XA_MEMPRIORITY_HIGHER
7	XA_MEMPRIORITY_CRITICAL

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.
- XA_API_FATAL_INVALID_CMD_TYPE
i_idx is an invalid memory block number; valid block numbers obey the relation $0 \leq i_idx < n_mems$ (See XA_API_CMD_GET_N_MEMTABS).

2.6.12 XA_API_CMD_SET_MEM_PTR

Table 2-27 XA_API_CMD_SET_MEM_PTR Command

Subcommand	Memory index
Description	This command passes the pointer to the allocated memory to the codec. The pointer is then stored in the memory tables structure allocated earlier. For the input and output buffers, running this command during the main codec loop is legitimate.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_SET_MEM_PTR i_idx Index of the memory. pv_value alloc – Pointer to the allocated memory buffer.
Restrictions	The pointer must be correctly aligned to the requirements.

Example

```
int index;
void * alloc; //alloc is a pointer to the aligned memory
res = (*api_func) (api_obj,
                  XA_API_CMD_SET_MEM_PTR,
                  index,
                  (pVOID) alloc);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.
- XA_API_FATAL_INVALID_CMD_TYPE
i_idx is an invalid memory block number; valid block numbers obey the relation $0 \leq i_idx < n_mems$ (See XA_API_CMD_GET_N_MEMTABS).
- XA_API_FATAL_MEM_ALIGN
pv_value is not of the required alignment for the requested memory block.

2.6.13 XA_API_CMD_INPUT_OVER

Table 2-28 XA_API_CMD_INPUT_OVER Command

Subcommand	None
Description	This command informs the codec that the end of the input data has been reached. This situation can arise both in the initialization and processing loop.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_INPUT_OVER i_idx NULL pv_value NULL
Restrictions	None

Example

```
res = (*api_func)(api_obj,  
    XA_API_CMD_INPUT_OVER,  
    0,  
    NULL);
```

Errors

- Common API Errors.

2.6.14 XA_API_CMD_SET_INPUT_BYTES

Table 2-29 XA_API_CMD_SET_INPUT_BYTES Command

Subcommand	None
Description	This command sets the number of bytes available in the input buffer for the codec. It is used both in the initialization and processing loop. It is the number of valid bytes from the buffer pointer. At least the minimum buffer size should be requested unless this is the end of the data.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_SET_INPUT_BYTES i_idx NULL pv_value &buff_size – Pointer to the input byte variable.
Restrictions	None

Example

```
int buff_size;  
res = (*api_func)(api_obj,  
    XA_API_CMD_SET_INPUT_BYTES,  
    0,  
    (pVOID) &buff_size);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

2.6.15 XA_API_CMD_GET_CURIDX_INPUT_BUF

Table 2-30 XA_API_CMD_GET_CURIDX_INPUT_BUF Command

Subcommand	None
Description	This command gets the number of input buffer bytes consumed by the codec. It is used both in the initialization and processing loop.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure i_cmd XA_API_CMD_GET_CURIDX_INPUT_BUF i_idx NULL pv_value &bytes_consumed – Pointer to the bytes consumed variable
Restrictions	None

Example

```
int bytes_consumed;  
res = (*api_func)(api_obj,  
    XA_API_CMD_GET_CURIDX_INPUT_BUF,  
    0,  
    (pVOID) &bytes_consumed);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

2.6.16 XA_API_CMD_EXECUTE

Table 2-31 XA_CMD_TYPE_DO_EXECUTE Subcommand

Subcommand	XA_CMD_TYPE_DO_EXECUTE
Description	This command runs the codec.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_EXECUTE i_idx XA_CMD_TYPE_DO_EXECUTE pv_value NULL
Restrictions	None

Example

```
res = (*api_func)(api_obj,  
                XA_API_CMD_EXECUTE,  
                XA_CMD_TYPE_DO_EXECUTE,  
                NULL);
```

Errors

- Common API Errors.
- See the codec-specific section for Process errors.

Table 2-32 XA_CMD_TYPE_DONE_QUERY Subcommand

Subcommand	XA_CMD_TYPE_DONE_QUERY
Description	This command checks to see if the processing is completed. If it has, the flag value is set to 1; otherwise, it is set to zero. The pointer to the flag is passed as an argument. Processing by the codec can continue for several invocations of the DO_EXECUTE command after the last input data has been passed to the codec. Thus the application should not assume that the codec has finished generating all its output until the command indicates it.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_EXECUTE i_idx XA_CMD_TYPE_DONE_QUERY pv_value &flag – Pointer to the flag variable.
Restrictions	None

Example

```
int flag;
res = (*api_func)(api_obj,
    XA_API_CMD_EXECUTE,
    XA_CMD_TYPE_DONE_QUERY,
    (pVOID) &flag);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

Table 2-33 XA_CMD_TYPE_DO_RUNTIME_INIT Subcommand

Subcommand	XA_CMD_TYPE_DO_RUNTIME_INIT
Description	This command resets the decoder's history buffers. It can be used to avoid distortions and clicks by facilitating playback ramping up and down during trick play. The command should be issued before the application starts feeding the decoder with new data from a random place in the input stream. Note: This command is available in API version 1.14 or later.
Actual Parameters	<code>p_xa_module_obj</code> <code>api_obj</code> – Pointer to API structure. <code>i_cmd</code> XA_API_CMD_EXECUTE <code>i_idx</code> XA_CMD_TYPE_DO_RUNTIME_INIT <code>pv_value</code> NULL
Restrictions	None.

Example

```
res = (*api_func)(api_obj,  
                 XA_API_CMD_EXECUTE,  
                 XA_CMD_TYPE_DO_RUNTIME_INIT,  
                 NULL);
```

Errors

- Common API Errors.

2.6.17 XA_API_CMD_GET_OUTPUT_BYTES

Table 2-34 XA_API_CMD_GET_OUTPUT_BYTES Command

Subcommand	None
Description	This command obtains the number of bytes output during the last codec processing.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_OUTPUT_BYTES i_idx NULL pv_value &out_bytes – Pointer to the output bytes variable.
Restrictions	None

Example

```
int out_bytes;
res = (*api_func)(api_obj,
    XA_API_CMD_GET_OUTPUT_BYTES,
    0,
    (pVOID) &out_bytes);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

2.6.18 XA_API_CMD_GET_CONFIG_PARAM

Table 2-35 XA_CONFIG_PARAM_CUR_INPUT_STREAM_POS Subcommand

Subcommand	XA_CONFIG_PARAM_CUR_INPUT_STREAM_POS
Description	This command reads the current input stream position, which is equal to the total number of consumed input bytes until the start of the input buffer. This running counter is set to zero at library initialization time and incremented every time the codec library consumes any bytes from the input buffer. If the application layer places a unit of input data with a byte size equal to <i>size</i> at byte offset in the input buffer, then the input stream position range for this unit may be calculated as follows: $\begin{aligned} \text{start_pos} &= \text{CUR_INPUT_STREAM_POS} + \text{offset} \\ \text{end_pos} &= \text{CUR_INPUT_STREAM_POS} + \text{offset} + \text{size} \end{aligned}$
Actual Parameters	<i>p_xa_module_obj</i> <i>api_obj</i> – Pointer to API structure. <i>i_cmd</i> XA_API_CMD_GET_CONFIG_PARAM <i>i_idx</i> XA_CONFIG_PARAM_CUR_INPUT_STREAM_POS <i>pv_value</i> &ui_cur_input_stream_pos – Pointer to the current input stream position variable.
Restrictions	The current input stream position counter is 32 bits and will, therefore, overflow and wrap around if the input stream length is more than $2^{32}-1$ bytes. This command is available in API version 1.15 or later.

Example

```
unsigned int ui_cur_input_stream_pos;
res = (*api_func)(api_obj,
    XA_API_CMD_GET_CONFIG_PARAM,
    XA_CONFIG_PARAM_CUR_INPUT_STREAM_POS,
    (void *) &ui_cur_input_stream_pos);
```

Errors

- Common API Errors.

Table 2-36 XA_CONFIG_PARAM_GEN_INPUT_STREAM_POS Subcommand

Subcommand	XA_CONFIG_PARAM_GEN_INPUT_STREAM_POS
Description	This command reads the input stream position of the unit (for example, frame) corresponding to the generated (decoded or encoded) output data block. That is, if the main processing (DO_EXECUTE) call into the library generates any data in the output buffer, then this command reads the total number of input bytes consumed until the start of the unit that has been processed and placed into the output buffer. For example, if the application layer places a unit in the input buffer at input stream position <code>start_pos</code> (see Table 2-35), when the library generates the decoded or encoded data corresponding to this unit, it sets <code>GEN_INPUT_STREAM_POS</code> to <code>start_pos</code> .
Actual Parameters	<code>p_xa_module_obj</code> <code>api_obj</code> – Pointer to API structure. <code>i_cmd</code> <code>XA_API_CMD_GET_CONFIG_PARAM</code> <code>i_idx</code> <code>XA_CONFIG_PARAM_GEN_INPUT_STREAM_POS</code> <code>pv_value</code> <code>&ui_gen_input_stream_pos</code> – Pointer to the input stream position of the generated data variable.
Restrictions	The input stream position of the generated data counter is 32 bits and will, therefore overflow and wrap-around, if the input stream length is more than $2^{32}-1$ bytes. This command is available in API version 1.15 or later.

Example

```
unsigned int ui_gen_input_stream_pos;
res = (*api_func)(api_obj,
                  XA_API_CMD_GET_CONFIG_PARAM,
                  XA_CONFIG_PARAM_GEN_INPUT_STREAM_POS,
                  (void *) &ui_gen_input_stream_pos);
```

Errors

- Common API Errors.

2.6.19 XA_API_CMD_SET_CONFIG_PARAM

Table 2-37 XA_CONFIG_PARAM_CUR_INPUT_STREAM_POS Subcommand

Subcommand	XA_CONFIG_PARAM_CUR_INPUT_STREAM_POS
Description	This command resets the current input stream position. For more information, see Table 2-35.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_CONFIG_PARAM_CUR_INPUT_STREAM_POS pv_value &ui_cur_input_stream_pos – Pointer to the current input stream position variable.
Restrictions	This command is available in API version 1.15 or later.

Example

```

unsigned int ui_cur_input_stream_pos = 0;
res = (*api_func)(api_obj,
                  XA_API_CMD_SET_CONFIG_PARAM,
                  XA_CONFIG_PARAM_CUR_INPUT_STREAM_POS,
                  (void *) &ui_cur_input_stream_pos);

```

Errors

Common API Errors.

3. HiFi DSP MP3 Decoder

The HiFi DSP MP3 Decoder conforms to the generic codec API. The following flowchart for the MP3 Decoder integration details the sequence of commands used on the example testbench.

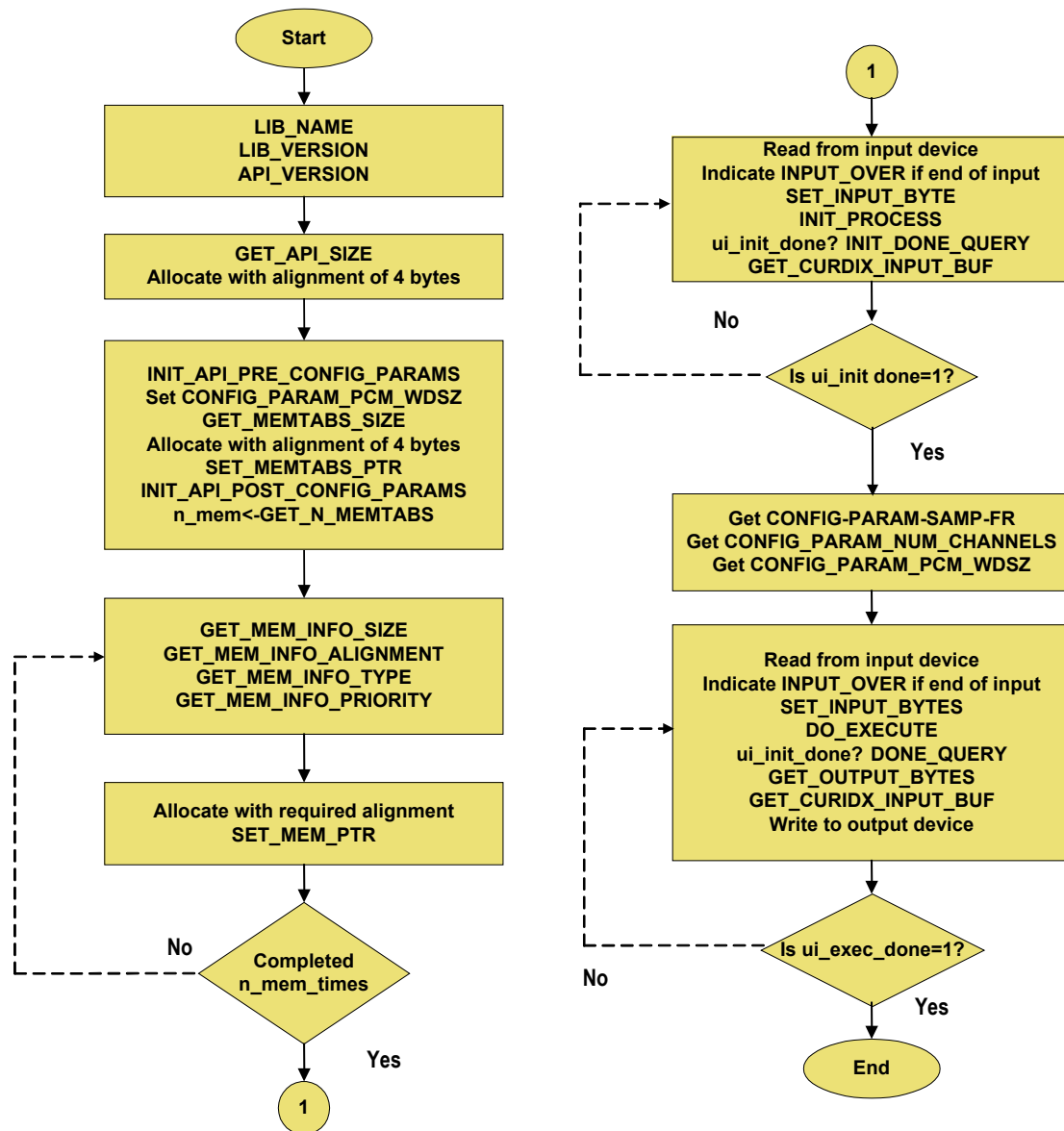


Figure 3 Flowchart for MP3 Decoder Integration

3.1 Files Specific to the MP3 Decoder

The MP3 decoder parameter header file (`include/mp3_dec`):

- `xa_mp3_dec_api.h`

The MP3 decoder libraries (`lib`):

- `xa_mp2_dec.a` (MP2 decoder)
- `xa_mp3_dec.a` (MP3 decoder)
- `xa_mp3mch_dec.a` (MP3 5.1-channel decoder)

The MP3 decoder API call is defined as:

```
XA_ERRORCODE xa_mp3_dec(xa_codec_handle_t p_xa_module_obj,
                        WORD32             i_cmd,
                        WORD32             i_idx,
                        pVOID              pv_value);
```

3.2 Configuration Parameters

The HiFi MP3 decode algorithm accepts the following parameters.

- **pcm_wd_sz** – The PCM word size requested.
 - Smaller or equal to 16 – The value used is 16.
 - Greater than 16 – The value used is 24.
- **crc_check** – Enables (non-zero) or disables (0) CRC validation for error-protected Layer I and II streams. If CRC validation is enabled and fails for an error-protected stream, a non-fatal error code is returned along with the decoded output. By default, CRC checking is disabled.
- **mch_enable** – Enables (non-zero) or disables (0) Layer II 5.1-channel decoding. This flag is supported by the MP3 5.1-channel Decoder only and is enabled by default.
- **dabmp2** – Enables (non-zero) or disables (0) Digital Audio Broadcasting (DAB)-specific Scale factor CRC (see *ETSI EN 300 401 V1.4.1 Radio Broadcasting Systems* ^[3]).

The parameters required for storage and audio output through a Digital to Analog Converter (DAC) are obtained from the bitstream during the initialization stage. After the initialization, the following parameters are available:

- **samp_freq** – This is the sample frequency of the output buffer samples. The units are Hz (samples per second). For example, a valid output is 44100.
- **num_chan** – This is the number of channels of data put into the output buffer.

- The stereo-only libraries (`mp2_dec` and `mp3_dec`) have only two valid values, 1 (mono) and 2 (stereo), for this parameter. In the case of stereo, the output data are interleaved in the output buffer, with the first output sample in each pair being the left-channel value and the second sample in each pair being the right-channel value.
- In the MP3 5.1-channel Decoder, the value of the `num_chan` parameter is between 1 and 6. The PCM samples for the separate channels are interleaved in the output buffer – the exact channel mapping can be retrieved through the `chan_map` parameter.

Note The `num_chan` parameter indicates the actual number of channels present in the output buffer. That is, unused PCM sample offsets will not be present in the output.

- **chmode_info** – This is the channel mode information:
 - bits 1:0 – 0 (stereo), 1 (joint stereo), 2 (dual mono), 3 (mono).
 - bit 4: intensity stereo, if set to 1.
 - bit 5: mid-side stereo, if set to 1.
- **bitrate** – This is the data rate (bit rate) of the input bitstream. The units are Kbps (kilobits per second). For example, a valid output is 320.
- **pcm_wd_sz** – This is the PCM word size of the data in the output buffer. The only values are 16 or 24. This is only useful if neither 16 nor 24 are given to the decoder.
- **mch_status** – This is the multi-channel status of the library and the input stream. Status codes:
 - `XA_MP3DEC_MCH_STATUS_UNSUPPORTED (0)` – the library does not support multi-channel decoding (that is, the library is either `mp2_dec` or `mp3_dec`).
 - `XA_MP3DEC_MCH_STATUS_DISABLED (1)` – multi-channel decoding is disabled through the `mch_enable` configuration parameter (`mp3mch_dec` only).
 - `XA_MP3DEC_MCH_STATUS_NOT_PRESENT (2)` – multi-channel decoding is supported, but the input stream is mono or stereo (`mp3mch_dec` only).
 - `XA_MP3DEC_MCH_STATUS_PRESENT (3)` – the library is decoding multi-channel content (`mp3mch_dec` only).
- **extn_present** – This indicates if an extension sub-stream is present in the input stream (`mp3mch_dec` only).
- **lfe_present** – This indicates if the LFE (low-frequency effects) channel is present in the input multi-channel stream (`mp3mch_dec` only).
- **num_xchan** – This is the number of extra full-bandwidth channels in addition to the main audio channels. This number does not include the LFE channel.
 - Valid values: 0 through 3.
 - For example, a 5.1-channel stream will have `num_xchan` set to 3 and `lfe_present` set to 1.
- **chan_config** – This specifies the channel configuration of the input stream (`mp3mch_dec` only). Valid values and channel configurations:
 - 1 – L(Left) / C (Center) / R (Right) / Ls (Left Surround) / Rs (Right Surround)

- 2 – L / C / R / Cs (Center Surround)
- 3 – L / C / R
- 4 – L / C / R / L2 (Second Left) / R2 (Second Right)
- 5 – L / R / Ls / Rs
- 6 – L / R / Cs
- 7 – L / R
- 8 – L / R / L2 / R2
- 9 – C
- 10 – C / L2 / R2
- **chan_map** – This parameter specifies how the channels are arranged in the output buffer.
 - Nibbles 0 to 5 of the `chan_map` parameter variable are set to channel index values based on their sample offsets in the interleaved output PCM buffer. If a channel (with channel index C) appears at sample offset N in the interleaved output PCM buffer, then the Nth nibble of the `chan_map` parameter is set to C.
 - Valid channel indices: 0 – L, 1 – C, 2 – R, 3 – Ls/Cs, 4 – Rs, 5 – L2, 6 – R2, 7 – LFE, 0 x F – invalid PCM sample offset.
 - For example, `chan_map = 0xFF743120` indicates that the left channel (channel index 0) is present at sample offset 0, the right channel (channel index 2) is present at sample offset 1, the center channel (channel index 1) is present at sample offset 2, the left surround (channel index 3) is present at sample offset 3, the right surround (channel index 4) is present at sample offset 4, and the LFE channel (channel index 7) is present at sample offset 5.

Note The values of the `num_chan`, `lfe_present`, `num_xchan`, and `chan_config` parameters can be derived from this parameter.

3.3 Usage Notes

Although the HiFi MP3 Decoder conforms to the generic codec API described in Section 2, the following notes must be considered to ensure correct decoder operation.

- The MP3 5.1-channel Decoder can process both base and extension input streams. If an extension stream is present, the input to the decoder must be a frame-by-frame concatenation of the base and extension streams:
 - [base frame 0] [extension frame 0] [base frame 1] [extension frame 1] ...
- The PCM samples for the separate channels are interleaved in the output buffer. The number of channels present in the output buffer depends on the number of channels present in the input stream. Unused PCM offsets are not present in the output buffer.
- For each `DO_EXECUTE` call, the MP3 Decoder expects enough data in the input buffer to parse and decode one complete frame. If the input buffer contains insufficient data, the MP3 Decoder does not consume any bytes and returns a non-fatal error that allows the application to fill in additional data before calling the decoder again.

Note The MP3 5.1-channel Decoder consumes both the base frame and the extension frame (if present) and produces the decoded audio data in a single `DO_EXECUTE` call.

- During initialization, the MP3 5.1-channel Decoder detects the availability of multi-channel content in the input stream by doing a CRC check on the base and extension frames (if present). If the CRC check fails, the MP3 5.1-channel Decoder indicates through the `mch_status` parameter that multi-channel content is not present and switches to 2-channel decoding only.

3.4 MP3 Decoder-specific Commands

This section describes commands unique to the HiFi MP3 Decoder. They are listed in sections based on their primary command type (`i_cmd`). Each section contains a table for every subcommand.

If no sub-commands are available, only one primary command is presented.

3.4.1 Initialization and Process Errors

The following errors can occur during initialization or running API calls:

- **XA_MP3DEC_CONFIG_NONFATAL_MCH_NOT_SUPPORTED**
The application tried to enable multi-channel support in the MP2 Decoder or the MP3 Decoder libraries. Layer II 5.1-channel decoding is supported only by the MP3 5.1-channel Decoder library.
- **XA_MP3DEC_CONFIG_NONFATAL_INVALID_GEN_STRM_POS**
The input stream position of the decoded data is invalid. This may occur if the decoder has not generated any output data, a runtime-init ramp-down frame is active, or the application resets the current input stream position.
- **XA_MP3DEC_CONFIG_NONFATAL_INVALID_PARAM_VALUE**
Invalid set parameter. This error occurs when the provided config parameter value is invalid but non-fatal.
- **XA_MP3DEC_EXECUTE_NONFATAL_NEED_MORE**
The codec requires more bits to continue the processing.
- **XA_MP3DEC_EXECUTE_NONFATAL_CANNOT_REWIND**
The MP3 bitstream syntax states that some bits corresponding to the current frame can be contained in the space designated for the previous frame. This error situation occurs when the bitstream needs to be rewound beyond the buffered bits.
- **XA_MP3DEC_EXECUTE_NONFATAL_CHANGED_CHANNELS**
The number of channels changed between successive frames.
- **XA_MP3DEC_EXECUTE_NONFATAL_CHANGED_SAMP_FREQ**
The sample frequency changed between successive frames.

- **XA_MP3DEC_EXECUTE_NONFATAL_CHANGED_LAYER**
The MPEG layer of the bitstream changed between successive frames.
- **XA_MP3DEC_EXECUTE_NONFATAL_NEXT_SYNC_NOT_FOUND**
The bytes required for the current frame exceeded the buffer limit.
- **XA_MP3DEC_EXECUTE_NONFATAL_CRC_FAILED**
CRC validation failed for an error-protected stream.
- **XA_MP3DEC_EXECUTE_NONFATAL_NO_MAIN_AUDIO_INPUT**
Insufficient data is present to decode the main audio data. This error may occur at broken frame boundaries or during trick play where input frames are not time-continuous.
- **XA_MP3DEC_EXECUTE_NONFATAL_MCH_CRC_ERROR**
The CRC check of the multi-channel frame failed during normal decoding. Only the stereo channels will be decoded, and the rest will be set to 0. This error can be returned only by the MP3 5.1-channel Decoder.
- **XA_MP3DEC_EXECUTE_NONFATAL_MCH_EXT_NOTFOUND**
The CRC check of the extension multi-channel frame failed during normal decoding. Only the stereo channels will be decoded, and the rest will be set to 0. This error can be returned only by the MP3 5.1-channel Decoder.
- **XA_MP3DEC_EXECUTE_NONFATAL_SCF_CRC_FAILED1**
The DAB-specific scale factor CRC check failed. One CRC is in error.
- **XA_MP3DEC_EXECUTE_NONFATAL_SCF_CRC_FAILED2**
The DAB-specific scale factor CRC check failed. Two CRCs are in error.
- **XA_MP3DEC_EXECUTE_NONFATAL_SCF_CRC_FAILED3**
The DAB-specific scale factor CRC check failed. Three CRCs are in error.
- **XA_MP3DEC_EXECUTE_NONFATAL_SCF_CRC_FAILED4**
The DAB-specific scale factor CRC check failed. Four CRCs are in error.
- **XA_MP3DEC_EXECUTE_NONFATAL_INVALID_BITRATE_MODE_COMB**
The MP3 bitstream syntax states that for layer II, all combinations of total bit rate and mode are not allowed.
- **XA_MP3DEC_CONFIG_NONFATAL_MP3_PCM_ADJUST_16**
The PCM value given is adjusted to 16.
- **XA_MP3DEC_CONFIG_NONFATAL_MP3_PCM_ADJUST_24**
The PCM value given is adjusted to 16.

Fatal errors require the codec to be completely reinitialized and presented with an alternative bitstream.

- `XA_MP3DEC_EXECUTE_FATAL_UNSUPPORTED_LAYER`
A bitstream is presented with an unsupported MPEG layer.
- `XA_MP3DEC_EXECUTE_FATAL_OVERLOADED_IN_BUF`
The decoder attempted to decoding the bitstream beyond the available data.
- `XA_MP3DEC_EXECUTE_FATAL_STREAM_ERROR`
Errors are encountered in the bitstream.

3.4.2 XA_API_CMD_SET_CONFIG_PARAM

Table 3-1 XA_MP3DEC_CONFIG_PARAM_PCM_WDSZ Subcommand

Subcommand	XA_MP3DEC_CONFIG_PARAM_PCM_WDSZ
Description	This command sets the PCM Word format size of the output samples in bits. The word format is set to 16 bits or 24 bits.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_MP3DEC_CONFIG_PARAM_PCM_WDSZ pv_value &pcm_wd_sz – Pointer to word size variable.
Restrictions	Only 16 and 24 are valid internal word format word sizes. 16, if pcm_wd_sz <= 16 24, if pcm_wd_sz > 16 See non-fatal errors below.

Note The 24-bit format is returned in 32-bit aligned words with the data left justified in the word. This is the reason that the buffer size requirement for 24-bit samples is double the requirement for 16-bit samples.

Example

```
int pcm_wd_sz = 24;
res = (*api_func)(api_obj,
                  XA_API_CMD_SET_CONFIG_PARAM,
                  XA_MP3DEC_CONFIG_PARAM_PCM_WDSZ,
                  (pVOID) &pcm_wd_sz);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.
- XA_MP3DEC_CONFIG_NONFATAL_MP3_PCM_ADJUST_16
The value given has been adjusted to 16.
- XA_MP3DEC_CONFIG_NONFATAL_MP3_PCM_ADJUST_24
The value given has been adjusted to 24.

Table 3-2 XA_MP3DEC_CONFIG_PARAM_CRC_CHECK Subcommand

Subcommand	XA_MP3DEC_CONFIG_PARAM_CRC_CHECK
Description	This command enables or disables CRC validation for error-protected Layer I and II streams. During the processing phase, CRC failures are signaled to the application through a non-fatal error code (see Section 3.4.1).
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_MP3DEC_CONFIG_PARAM_CRC_CHECK pv_value &crc_check – Pointer to CRC check variable.
Restrictions	0 – disables CRC validation (default); non-zero – enables CRC validation.

Example

```
int crc_check = 1;
res = (*api_func) (api_obj,
                  XA_API_CMD_SET_CONFIG_PARAM,
                  XA_MP3DEC_CONFIG_PARAM_CRC_CHECK,
                  (pVOID) &crc_check);
```

Errors

- Common API Errors.

Table 3-3 XA_MP3DEC_CONFIG_PARAM_MCH_ENABLE Subcommand

Subcommand	XA_MP3DEC_CONFIG_PARAM_MCH_ENABLE
Description	This command enables or disables multi-channel decoding in the MP3 5.1-channel Decoder library.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_MP3DEC_CONFIG_PARAM_MCH_ENABLE pv_value &mch_enable – Pointer to multi-channel enable flag.
Restrictions	This flag is supported by the MP3 5.1-channel Decoder library only. 0 – disables multi-channel decoding; non-zero – enables multi-channel decoding (default).

Example

```
int mch_enable = 0;
res = (*api_func)(api_obj,
                  XA_API_CMD_SET_CONFIG_PARAM,
                  XA_MP3DEC_CONFIG_PARAM_MCH_ENABLE,
                  (pVOID) &mch_enable);
```

Errors

- Common API Errors.

Table 3-4 XA_MP3DEC_CONFIG_PARAM_DAB_MP2 Subcommand

Subcommand	XA_MP3DEC_CONFIG_PARAM_DAB_MP2
Description	This command enables or disables DAB-specific Scale Factor CRC checking.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_MP3DEC_CONFIG_PARAM_DAB_MP2 pv_value & dabmp2 – Pointer to dabmp2 enable flag.
Restrictions	0 – disables DAB-specific Scale Factor CRC validation (default); non-zero – enables DAB-specific Scale Factor CRC validation.

Example

```
int dabmp2 = 0;
res = (*api_func)(api_obj,
                  XA_API_CMD_SET_CONFIG_PARAM,
                  XA_MP3DEC_CONFIG_PARAM_DAB_MP2,
                  (pVOID) &dabmp2);
```

Errors

- Common API Errors.

Table 3-5 XA_MP3DEC_CONFIG_PARAM_ACTIVATE_VLC_REWIND Subcommand

Subcommand	XA_MP3DEC_CONFIG_PARAM_ACTIVATE_VLC_REWIND
Description	This command enables or disables VLC rewind. This method is used to recover from a bitstream overrun caused by a bitstream error, during the VLC decoding process. If this feature is disabled, the decoder will drop the frame.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_MP3DEC_CONFIG_PARAM_ACTIVATE_VLC_REWIND pv_value & activate_vlc_rewind – Pointer to activate_vlc_rewind flag.
Restrictions	0 – disable (default). 1 – enable.

Example

```
Int activate_vlc_rewind = 0;
res = (*api_func)(api_obj,
                  XA_API_CMD_SET_CONFIG_PARAM,
                  XA_MP3DEC_CONFIG_PARAM_ACTIVATE_VLC_REWIND,
                  (pVOID) & activate_vlc_rewind);
```

Errors

- Common API Errors.

Table 3-6 XA_MP3DEC_CONFIG_PARAM_NONSTD_STRM_SUPPORT Subcommand

Subcommand	XA_MP3DEC_CONFIG_PARAM_NONSTD_STRM_SUPPORT
Description	This command enables or disables non-standard stream support by relaxing the standard validity checks for streams.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_SET_CONFIG_PARAM i_idx XA_MP3DEC_CONFIG_PARAM_NONSTD_STRM_SUPPORT pv_value & activate_nonstd – Pointer to activate_nonstd flag.
Restrictions	0 – disable (default). 1 – enable.

Example

```

Int activate_nonstd = 0;
res = (*api_func)(api_obj,
                  XA_API_CMD_SET_CONFIG_PARAM,
                  XA_MP3DEC_CONFIG_PARAM_NONSTD_STRM_SUPPORT,
                  (pVOID) & activate_nonstd);

```

Errors

- Common API Errors.
- XA_MP3DEC_CONFIG_NONFATAL_INVALID_PARAM_VALUE

3.4.3 XA_API_CMD_GET_CONFIG_PARAM

Table 3-7 XA_MP3DEC_CONFIG_PARAM_SAMP_FREQ Subcommand

Subcommand	XA_MP3DEC_CONFIG_PARAM_SAMP_FREQ
Description	This command fetches the sampling frequency from the codec. The sampling frequency can be used, for example, to write a '.wav' file header or to initialize a DAC or sample rate converter for real-time playback. The sampling frequency (in Hz) is returned.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_MP3DEC_CONFIG_PARAM_SAMP_FREQ pv_value &samp_freq – Pointer to sampling frequency variable.
Restrictions	None

Example

```
int samp_freq;
res = (*api_func)(api_obj,
                  XA_API_CMD_GET_CONFIG_PARAM,
                  XA_MP3DEC_CONFIG_PARAM_SAMP_FREQ,
                  (pVOID) &samp_freq);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

Table 3-8 XA_MP3DEC_CONFIG_PARAM_NUM_CHANNELS Subcommand

Subcommand	XA_MP3DEC_CONFIG_PARAM_NUM_CHANNELS
Description	This command fetches the number of channels in the decoded bitstream. This information can be used to write a '.wav' file header or to initialize a DAC or sample rate converter for real-time playout.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_MP3DEC_CONFIG_PARAM_NUM_CHANNELS pv_value &num_chan – Pointer to the number of channels.
Restrictions	None

Example

```
int num_chan;
res = (*api_func)(api_obj,
                  XA_API_CMD_GET_CONFIG_PARAM,
                  XA_MP3DEC_CONFIG_PARAM_NUM_CHANNELS,
                  (pVOID) &num_chan);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC

pv_value is NULL.

Table 3-9 XA_MP3DEC_CONFIG_PARAM_CHMODE_INFO Subcommand

Subcommand	XA_MP3DEC_CONFIG_PARAM_CHMODE_INFO
Description	This command gets the channel mode information present in the frame header. The information can be used to derive the channel mode (nibble 0) and the mode extension values (nibble 1). The channel mode information is decoded as specified: Bits 1:0 – 0 (stereo), 1 (joint stereo), 2 (dual mono), 3 (mono) Bit 4 – intensity stereo, if set to 1 Bit 5 – mid-side stereo, if set to 1
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_MP3DEC_CONFIG_PARAM_CHMODE_INFO pv_value &chmode_info – Pointer to the channel mode variable.
Restrictions	None

Example

```
int chmode_info;
res = (*api_func) (api_obj,
                  XA_API_CMD_GET_CONFIG_PARAM,
                  XA_MP3DEC_CONFIG_PARAM_CHMODE_INFO,
                  (pVOID) &chmode_info);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

Table 3-10 XA_MP3DEC_CONFIG_PARAM_BITRATE Subcommand

Subcommand	XA_MP3DEC_CONFIG_PARAM_BITRATE
Description	This command fetches the input stream's data rate (bit rate) in Kbps.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_MP3DEC_CONFIG_PARAM_BITRATE pv_value &bitrate – Pointer to bit rate variable.
Restrictions	None

Example

```
int bitrate;
res = (*api_func) (api_obj,
                  XA_API_CMD_GET_CONFIG_PARAM,
                  XA_MP3DEC_CONFIG_PARAM_BITRATE,
                  (pVOID) &bitrate);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

Table 3-11 XA_MP3DEC_CONFIG_PARAM_PCM_WDSZ Subcommand

Subcommand	XA_MP3DEC_CONFIG_PARAM_PCM_WDSZ
Description	This command fetches the output PCM word size in bits, which may be different from the one given; if the user-specified value is other than 16 or 24.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_MP3DEC_CONFIG_PARAM_PCM_WDSZ pv_value &pcm_wd_sz – Pointer to the PCM word size.
Restrictions	None

Example

```
int pcm_wd_sz;
res = (*api_func)(api_obj,
                  XA_API_CMD_GET_CONFIG_PARAM,
                  XA_MP3DEC_CONFIG_PARAM_PCM_WDSZ,
                  (pVOID) &pcm_wd_sz);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

Table 3-12 XA_MP3DEC_CONFIG_PARAM_MCH_STATUS Subcommand

Subcommand	XA_MP3DEC_CONFIG_PARAM_MCH_STATUS
Description	This command gets the multi-channel status of the library and the input stream. The status may be one of the following: 0 – multi-channel decoding not supported. 1 – multi-channel decoding disabled. 2 – multi-channel content not available. 3 – multi-channel content decoding in progress. See Section 3.2.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_MP3DEC_CONFIG_PARAM_MCH_STATUS pv_value &mch_status – Pointer to the multi-channel status variable.
Restrictions	None

Example

```
int mch_status;
res = (*api_func)(api_obj,
                  XA_API_CMD_GET_CONFIG_PARAM,
                  XA_MP3DEC_CONFIG_PARAM_MCH_STATUS,
                  (pVOID) &mch_status);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

Table 3-13 XA_MP3DEC_CONFIG_PARAM_EXTN_PRESENT Subcommand

Subcommand	XA_MP3DEC_CONFIG_PARAM_EXTN_PRESENT
Description	This command indicates if an extension sub stream is present in the input stream. Only the MP3 5.1-channel Decoder supports this command.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_MP3DEC_CONFIG_PARAM_EXTN_PRESENT pv_value &extn_present – Pointer to the extension sub stream present flag.
Restrictions	None

Example

```
int extn_present;  
res = (*api_func) (api_obj,  
                  XA_API_CMD_GET_CONFIG_PARAM,  
                  XA_MP3DEC_CONFIG_PARAM_EXTN_PRESENT,  
                  (pVOID) &extn_present);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

Table 3-14 XA_MP3DEC_CONFIG_PARAM_LFE_PRESENT Subcommand

Subcommand	XA_MP3DEC_CONFIG_PARAM_LFE_PRESENT
Description	This command indicates if the LFE channel is present in the input multi-channel stream. Only the MP3 5.1-channel Decoder supports this command.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_MP3DEC_CONFIG_PARAM_LFE_PRESENT pv_value &lfe_present – Pointer to the LFE present flag.
Restrictions	None

Example

```
int lfe_present;
res = (*api_func)(api_obj,
                  XA_API_CMD_GET_CONFIG_PARAM,
                  XA_MP3DEC_CONFIG_PARAM_LFE_PRESENT,
                  (pVOID) &lfe_present);
```

Errors

- Common API Errors
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

Table 3-15 XA_MP3DEC_CONFIG_PARAM_NUM_XCHANS Subcommand

Subcommand	XA_MP3DEC_CONFIG_PARAM_NUM_XCHANS
Description	This command fetches the number of extra channels in addition to the main audio channels and, potentially, the LFE channel. Only the MP3 5.1-channel Decoder supports this command.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_MP3DEC_CONFIG_PARAM_NUM_XCHANS pv_value &num_xchan – Pointer to the number of extra channels' variable.
Restrictions	None

Example

```
int num_xchan;
res = (*api_func)(api_obj,
                  XA_API_CMD_GET_CONFIG_PARAM,
                  XA_MP3DEC_CONFIG_PARAM_NUM_XCHANS,
                  (pVOID) &num_xchan);
```

Errors

- Common API Errors
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

Table 3-16 XA_MP3DEC_CONFIG_PARAM_CHAN_CONFIG Subcommand

Subcommand	XA_MP3DEC_CONFIG_PARAM_CHAN_CONFIG
Description	This command fetches the input stream's channel configuration. Section 3.2 lists the valid channel configuration codes. Only the MP3 5.1-channel Decoder supports this command.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_MP3DEC_CONFIG_PARAM_CHAN_CONFIG pv_value &chan_config – Pointer to the channel configuration variable.
Restrictions	None

Example

```
int chan_config;
res = (*api_func) (api_obj,
                  XA_API_CMD_GET_CONFIG_PARAM,
                  XA_MP3DEC_CONFIG_PARAM_CHAN_CONFIG,
                  (pVOID) &chan_config);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

Table 3-17 XA_MP3DEC_CONFIG_PARAM_CHAN_MAP Subcommand

Subcommand	XA_MP3DEC_CONFIG_PARAM_CHAN_MAP
Description	This command fetches the channel map specifying details about the channel arrangement in the output buffer. See Section 3.2.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_MP3DEC_CONFIG_PARAM_CHAN_MAP pv_value &chan_map – Pointer to the channel map variable.
Restrictions	None

Example

```
int chan_map;
res = (*api_func) (api_obj,
                  XA_API_CMD_GET_CONFIG_PARAM,
                  XA_MP3DEC_CONFIG_PARAM_CHAN_MAP,
                  (pVOID) &chan_map);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

Table 3-18 XA_MP3DEC_CONFIG_PARAM_DAB_MP2 Subcommand

Subcommand	XA_MP3DEC_CONFIG_PARAM_DAB_MP2
Description	This command fetches the dabmp2 flag. See Section 3.2.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_MP3DEC_CONFIG_PARAM_DAB_MP2 pv_value &dabmp2 – Pointer to the channel map variable.
Restrictions	None

Example

```
int dabmp2;
res = (*api_func)(api_obj,
                  XA_API_CMD_GET_CONFIG_PARAM,
                  XA_MP3DEC_CONFIG_PARAM_DAB_MP2,
                  (pVOID) &dabmp2);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

Table 3-19 XA_MP3DEC_CONFIG_PARAM_ORIGINAL_OR_COPY Subcommand

Subcommand	XA_MP3DEC_CONFIG_PARAM_ORIGINAL_OR_COPY
Description	This command fetches the original/copy flag encoded in the frame header. 1 – The bitstream is original. 0 – The bitstream is a copy.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_MP3DEC_CONFIG_PARAM_ORIGINAL_OR_COPY pv_value &IsOriginal – Pointer to the flag variable.
Restrictions	This API returns the value read from the recently decoded frame header. Note This API should be called only after successful decoding of a frame. The return value may not be valid if a frame decoding error occurs.

Example

```
int IsOriginal;
res = (*api_func)(api_obj,
                  XA_API_CMD_GET_CONFIG_PARAM,
                  XA_MP3DEC_CONFIG_PARAM_ORIGINAL_OR_COPY,
                  (pVOID) &IsOriginal);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

Table 3-20 XA_MP3DEC_CONFIG_PARAM_COPYRIGHT_FLAG Subcommand

Subcommand	XA_MP3DEC_CONFIG_PARAM_COPYRIGHT_FLAG
Description	This command gets the copyright flag encoded in the bitstream 1 – The bitstream is copyright-protected. 0 – There is no copyright in the stream.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_MP3DEC_CONFIG_PARAM_COPYRIGHT_FLAG pv_value &IsCopyRigthProtected – Pointer to the flag variable.
Restrictions	This API returns the value read from the recently decoded frame header. Note This API should be called only after successful decoding of a frame. The return value may not be valid if a frame decoding error occurs.

Example

```
int IsCopyRigthProtected;
res = (*api_func)(api_obj,
                  XA_API_CMD_GET_CONFIG_PARAM,
                  XA_MP3DEC_CONFIG_PARAM_COPYRIGHT_FLAG,
                  (pVOID) &IsCopyRigthProtected);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

Table 3-21 XA_MP3DEC_CONFIG_PARAM_MCH_EXT_HDR_INFO Subcommand

Subcommand	XA_MP3DEC_CONFIG_PARAM_MCH_EXT_HDR_INFO
Description	This command returns the header information read from the multi-channel extension header. This information contains downmixing procedures, multi-channel audio playback information, and information related to multilingual audio, among other things. Note The interpretation of this information is available in ISO/IEC 13818 - part 3.
Actual Parameters	p_xa_module_obj api_obj – Pointer to API structure. i_cmd XA_API_CMD_GET_CONFIG_PARAM i_idx XA_MP3DEC_CONFIG_PARAM_MCH_EXT_HDR_INFO pv_value &mchExtHdrInfo info – Pointer to the structure of type. xa_mch_ext_hdr_info_t into which multi-channel extension header information is stored. The definition of the structure is present in xa_mp3_dec_api.h
Restrictions	This API returns the value read from the `recently decoded frame header`. Note This API should be called only after successful decoding of a frame. Note The return value may not be valid if a frame decoding error occurs. This API is available only in mp3mch_dec.

Example

```

xa_mch_ext_hdr_info_t mchExtHdrInfo;
res = (*api_func)(api_obj,
                  XA_API_CMD_GET_CONFIG_PARAM,
                  XA_MP3DEC_CONFIG_PARAM_MCH_EXT_HDR_INFO,
                  (pVOID) &mchExtHdrInfo);

```

Errors

- Common API Errors.

- XA_API_FATAL_MEM_ALLOC

`pv_value` is NULL.

Table 3-22 XA_MP3DEC_CONFIG_PARAM_MCH_COPYRIGHT_ID_PTR Subcommand

Subcommand	XA_MP3DEC_CONFIG_PARAM_MCH_COPYRIGHT_ID_PTR
Description	This command returns a pointer to an array of nine unsigned characters (representing 72 bits of copyright identification information received from the bitstream). The first bit is in bit 7 of <code>p_copyright_info[0]</code> , and the last bit is in bit 0 of <code>p_copyright_info[8]</code> .
Actual Parameters	<code>p_xa_module_obj</code> <code>api_obj</code> – Pointer to API structure. <code>i_cmd</code> XA_API_CMD_GET_CONFIG_PARAM <code>i_idx</code> XA_MP3DEC_CONFIG_PARAM_MCH_COPYRIGHT_ID_PTR <code>pv_value</code> <code>&p_copyright_info</code> – Pointer to the <code>copyright_info</code> array.
Restrictions	Note The application should treat the array containing the copyright identification as read-only. As the information is available only after accumulating 72 bits, one from each frame, the application will get a valid pointer only after successful accumulation. When no copyright identification is available in the stream, this API returns a NULL pointer. Also, when the accumulation is in progress, and no prior valid identifier is available, this API returns a NULL pointer. The return value may not be valid if a frame decoding error occurs. This API is available only in <code>mp3mch_dec</code>.

Example

```
char * p_copyright_info;  
res = (*api_func)(api_obj,  
                 XA_API_CMD_GET_CONFIG_PARAM,  
                 XA_MP3DEC_CONFIG_PARAM_MCH_COPYRIGHT_ID_PTR,  
                 (pVOID) &p_copyright_info);
```

Errors

- Common API Errors.
- XA_API_FATAL_MEM_ALLOC
pv_value is NULL.

4. Introduction to the Example Testbench

The supplied testbench consists of the following files:

- Testbench source files in `test/src`:
 - `xa_mp3_dec_error_handler.c`
 - `xa_mp3_dec_sample_testbench.c`
 - `id3_tag_decode.c`
- Optional post-processing testbench source files:
 - See `readme.txt` in `test/build`
- Test bench include files in `test/include`:
 - `id3_tag_decode.h`
- Makefile to build in `test/build`:
 - `makefile_testbench_sample`
- Sample parameter file to use during run in `test/build`:
 - `paramfilesimple.txt`

4.1 Making Executable Files

Build Decoder Testbench

To build the decoder testbench, follow these steps:

1. In the command prompt window, navigate to the `test/build` directory.
2. Enter the following command:
`xt-make -f makefile_testbench_sample clean <lib>`

Where,

`<lib>` is `mp2`, `mp3`, or `mp3mch`.

The `xa_<lib>_dec_test` example decoder is built.

Build Library

Note Optional post-processing can be built from the testbench source files. See `readme.txt` located at `test/build`.

Note If you have source code distribution, you must build the `<lib>` library before you build the testbench.

To build the library, follow these steps:

1. In the command prompt window, navigate to the `build` directory.
2. Enter the following command:

```
$xt-make clean <lib> install
```

The `<lib>` library is built and copied to the `lib` directory.

4.2 Usage

You can run the sample application using the direct command line options or with a parameter file.

The command line usage is as follows:

```
xt-run xa_<lib>_dec_test \
    -ifile:<infile> -ofile:<outfile> \
    [-pcmsz:<pcmbitwidth>] \
    [-crc:<crc_check>]
    [-mch:<mch_enable>]
    [-nonstd:<nonstd_enable>]
```

Where,

`<infile>` is the name of the input MPEG audio file.

`<outfile>` is the name of the output file.

`<pcmbitwidth>` is the PCM bit-width which can be either 16 or 24 bits.

`<crc_check>` is 0 to disable or 1 to enable CRC validation.

`<mch_enable>` is 0 to disable or 1 to enable multi-channel decoding (`mp3mch_dec` only).

`<nonstd_enable>` is 0 to disable (default) or 1 to enable the standard validity checks for streams. The following command-line options are specific to error concealment (not supported by default).

```
-dabmp2:<dabmp2_enable>
-numch:<numch>
-numsamps_perch:<numsamps_perch>

-num_mute_frames:<num_mute_frames>
```

```

-num_rampdown_samps:<num_rampdown_samps>
-num_rampup_samps:<num_rampup_samps>

-alpha0x:<alpha>
-beta0x:<beta>
-force_mute:<force_mute>Where
<numch> Number of PCM channels; Its value can be 1 or 2.
<numsamps_perch> Number of PCM samples per channel; Its value can be <= 1152.
<num_mute_frames> Number of frames to continue mute state during concealment phase.
<num_rampdown_samps> Number of samples to apply ramp-down while entering the concealment
phase.
<num_rampup_samps> Number of samples to apply ramp-up while exiting the concealment phase.
<alpha> Ramp down the scale factor in 1.31 format.
<beta> Ramp up the scale factor in 1.31 format.
<force_mute> 0 to disable; 1 to enable soft muting.
When set to 1, error concealment will forcefully activate and continue till this flag is set to zero.

```

If no command-line argument is provided, the application reads the commands from the paramfilesimple.txt parameter file in the same directory as the run.

The syntax for writing into the paramfilesimple.txt file is as follows:

```

@Start
@Input_path <path to be prepended to all input filenames>
@Output_path <path to be prepended to all output filenames>
<command line 1>
<command line 2>
....
@stop

```

The MP3 Decoder can be run for multiple test files using different command lines. The syntax for command lines in the parameter file is the same as the syntax for specifying options on the command line to the testbench program.

Note	Generally, 16-bit output is used for applications.
Note	All the @<command>s should be at the first column of a line except the @New_line command.
Note	All the @<command>s are case sensitive. If the command line in the parameter file has to be broken into two parts on two different lines, use the @New_line command. For example:
	<pre> <command line part 1> @New_line <command line part 2> </pre>
Note	Blank lines are ignored.
Note	Individual lines can be commented out using “//” at the beginning of the line.

4.3 ID3 Decoding Example

Part of the MP3 decoder example test bench is a decoder for the ID3 container format. This container is used to carry information about the content of the encoded audio, which is added to the bitstream. The two popular versions are ID3v1 and ID3v2. This section introduces these versions.

ID3v1

ID3v1 is placed towards the end of the MP3 file. For easy detection, a fixed size of 128 bytes was chosen. The tag has the structure as shown in Table 4-1.

Table 4-1 ID3v1 Structure

Field	Size (in Bytes)
"TAG"	3
Song title	30
Artist	30
Album	30
Year	4
Comment	30
Genre	1

The first three bytes are always "TAG" and is the identification that this is an ID3 tag. The easiest way to decode an ID3v1 tag is to look for the word "TAG" 128 bytes from the end of a file.

ID3v2

ID3v2 is a general tagging format for audio, which makes it possible to store meta-data about the audio inside the audio file itself. The ID3 tag described in this document is mainly targeted at files encoded with MPEG-1/2 layer I, MPEG-1/2 layer II, MPEG-1/2 layer III, and MPEG-2.5, but may work with other types of encoded audio or as a standalone format for audio meta-data.

ID3v2 is designed to be as flexible and expandable as possible to meet any new metadata information needs that might arise. To achieve this flexibility, ID3v2 is constructed as a container for several information blocks, called frames, whose format need not be known to the software that encounters them. At the start of every frame is a unique and predefined identifier, a size descriptor that allows software to skip unknown frames, and a flags field.

The overall tag structure for ID3v2 is as shown in Table 4-2.

Table 4-2 ID3v2 Structure Header (10 bytes)

ID3v2
Extended Header
Frames (variable length)
Padding
(variable length, OPTIONAL)
Footer (10 bytes, OPTIONAL)

5. References

- [1] *ISO/IEC 11172-3 Information technology -- Coding of moving pictures and associated audio for digital storage media at up to about 1,5 Mbit/s -- Part 3: Audio (MPEG-1)*
- [2] *ISO/IEC 13818-3 Information technology -- Generic coding of moving pictures and associated audio information -- Part 3: Audio (MPEG-2)*
- [3] *ETSI EN 300 401 V1.4.1 Radio Broadcasting Systems; Digital Audio Broadcasting (DAB) to mobile, portable, and fixed receivers*